

HANDBOOK

Polymind V2

RL-Enhanced Deep Reading AI System (Colab-Optimized Edition)

A production-minded guide to building a Colab-native deep reading system with retrieval engineering, bandit-based answer selection, conversational memory, and restart-safe artifacts.

This edition is formatted as a long-form handbook rather than a plain export. It keeps the full 23-section build path, the integrated debugging and experimentation material, and the appendix reference layers while presenting them in a cleaner print-first layout.

Shivashant

April 2026

About This Edition

Rendered from the canonical Markdown handbook with Pandoc and Tectonic using a custom book template, serif body typography, sans-serif navigational hierarchy, and boxed code treatment tuned for wide Colab-ready examples.

Contents

1	How To Use This Handbook	2
2	Build Map And Artifact Glossary	4
3	1. Introduction	6
3.1	Intuition	6
3.2	Colab-ready code cell	6
3.3	Expected output	7
3.4	Debugging tips	7
4	2. System Overview	8
4.1	Intuition	8
4.2	Colab-ready code cell	8
4.3	Expected output	9
4.4	Debugging tips	9
5	3. Colab Setup	10
5.1	Intuition	10
5.2	Colab-ready code cell	10
5.3	Expected output	11
5.4	Debugging tips	11
6	4. Project Structure	12
6.1	Intuition	12
6.2	Colab-ready code cell	12
6.3	Expected output	13
6.4	Debugging tips	13
7	5. Book Ingestion	14
7.1	Intuition	14
7.2	Colab-ready code cell	14
7.3	Expected output	15
7.4	Debugging tips	15
8	6. Text Cleaning	16
8.1	Intuition	16
8.2	Colab-ready code cell	16
8.3	Expected output	16

8.4	Debugging tips	17
9	7. Chapter Detection	18
9.1	Intuition	18
9.2	Colab-ready code cell	18
9.3	Expected output	19
9.4	Debugging tips	19
10	8. Semantic Chunking	20
10.1	Intuition	20
10.2	Colab-ready code cell	20
10.3	Expected output	20
10.4	Debugging tips	21
11	9. Chunk Quality Inspection	22
11.1	Intuition	22
11.2	Colab-ready code cell	22
11.3	Expected output	23
11.4	Debugging tips	23
12	10. Embeddings	24
12.1	Intuition	24
12.2	Colab-ready code cell	24
12.3	Expected output	24
12.4	Debugging tips	25
13	11. FAISS Index	26
13.1	Intuition	26
13.2	Colab-ready code cell	26
13.3	Expected output	26
13.4	Debugging tips	27
14	12. Retrieval System	28
14.1	Intuition	28
14.2	Colab-ready code cell	28
14.3	Expected output	29
14.4	Debugging tips	29
15	13. RAG Pipeline	30
15.1	Intuition	30
15.2	Colab-ready code cell	30
15.3	Expected output	31

15.4	Debugging tips	31
16	14. Generator Model	32
16.1	Intuition	32
16.2	Colab-ready code cell	32
16.3	Expected output	32
16.4	Debugging tips	33
17	15. Answer Generation	34
17.1	Intuition	34
17.2	Colab-ready code cell	34
17.3	Expected output	35
17.4	Debugging tips	35
18	16. Reward Function	36
18.1	Intuition	36
18.2	Colab-ready code cell	36
18.3	Expected output	36
18.4	Debugging tips	37
19	17. RL Bandit Layer	38
19.1	Intuition	38
19.2	Colab-ready code cell	38
19.3	Expected output	39
19.4	Debugging tips	39
20	18. Memory System	40
20.1	Intuition	40
20.2	Colab-ready code cell	40
20.3	Expected output	41
20.4	Debugging tips	41
21	19. System Assembly	42
21.1	Intuition	42
21.2	Colab-ready code cell	42
21.3	Expected output	42
21.4	Debugging tips	43
22	20. Persistence & Reloading	44
22.1	Intuition	44
22.2	Colab-ready code cell	44
22.3	Expected output	45

22.4	Debugging tips	45
23	21. Evaluation Framework	46
23.1	Intuition	46
23.2	Colab-ready code cell	47
23.3	Expected output	48
23.4	Debugging tips	48
24	22. Optimization & Scaling	49
24.1	Intuition	49
24.2	Colab-ready code cell	49
24.3	Expected output	50
24.4	Debugging tips	50
25	23. Future Extensions	51
25.1	Intuition	51
25.2	Colab-ready code cell	51
25.3	Expected output	51
25.4	Debugging tips	52
26	Appendix A. Mastery Casebook And Diagnostic Walkthrough	53
26.1	A.1 Why Mastery.pdf Is A Good Stress Test	53
26.2	A.2 Front-Matter Chapter False Positives	54
26.3	A.3 Abbreviation Splitting Around Dr. And U.S.	54
26.4	A.4 Same-Chapter Retrieval Collapse Without Diversity	55
26.5	A.5 Citation Hallucination Before Verification	56
26.6	A.6 Bandit Preference Drift When Reward Normalization Is Wrong	57
27	Appendix B. Benchmark And Experiment Cookbook	58
27.1	B.1 Building A Useful Mastery Benchmark Set	58
27.2	B.2 Running A Top-k Sweep	59
27.3	B.3 Running A Chunk-Size Sweep	59
27.4	B.4 Measuring Temperature Versus Reward	60
27.5	B.5 Comparing flan-t5-base And flan-t5-xl	60
27.6	B.6 Retrieval Ablations	61
27.7	B.7 Cached-Versus-Cold Profiling	61
28	Appendix C. Failure Playbooks And Recovery Paths	63
28.1	C.1 Ingestion And Chaptering Failures	63
28.2	C.2 Chunking And Offset Failures	63
28.3	C.3 Embeddings, FAISS, And Resume Failures	64

28.4	C.4 Retrieval Failures	64
28.5	C.5 Generation And Citation Failures	65
28.6	C.6 RL And Memory Failures	65
29	Appendix D. Artifact, API, And Notebook Reference	67
29.1	D.1 Artifact Contracts	67
29.2	D.2 Dataclass Reference	67
29.3	D.3 PolymindSystem Call Patterns	68
29.4	D.4 Notebook And Guide Synchronization	68
29.5	D.5 Guide Maintenance Checklist	69
30	Appendix E. Colab Operations, Cost, And Production Readiness	70
30.1	E.1 Runtime Profiles And What They Mean	70
30.2	E.2 Memory Hygiene On A T4	70
30.3	E.3 Cost Awareness Without Paralysis	71
30.4	E.4 Session Reset Playbook	71
30.5	E.5 Production Readiness Checklist	72
30.6	E.6 Scaling Beyond One Book	72
30.7	E.7 Final Review Checklist Before Shipping	73
31	Appendix F. Extended Foundations And Architecture Notes	74
31.1	F.1 Reading-system foundations	74
31.2	F.2 Architecture and Colab framing	76
31.3	F.3 Model and transformer background	77
32	Appendix G. Extended Retrieval, Generation, And RL Notes	79
32.1	G.1 Book processing and chunking details	79
32.2	G.2 Embeddings, FAISS, and retrieval details	80
32.3	G.3 Generation, reward, and policy details	81
33	Appendix H. Extended Evaluation, Optimization, And Assembly Notes	84
33.1	H.1 Evaluation and acceptance guidance	84
33.2	H.2 Optimization and future improvements	85
33.3	H.3 Final assembly and workflow recap	86

Polymind V2 is a production-minded deep reading system built for one demanding but realistic task: take a full book, turn it into a structured knowledge substrate, retrieve the right evidence under tight compute limits, generate grounded answers, choose the best answer with a bandit layer, and preserve conversational continuity across long study sessions. The design target is Google Colab with Drive-backed persistence, limited GPU memory, restart safety, and teaching clarity.

The pipeline is deliberately layered:

Book -> Clean -> Structure -> Chunk -> Embed -> Index -> Retrieve -> Generate -> Score
-> Select -> Memory

This handbook is source-first. Every major idea is paired with Colab-ready code that maps directly to the `polymind/` package and the notebook in `notebooks/polymind_v2_colab.ipynb`. The guide assumes `Mastery.pdf` is the concrete walkthrough book, but every module is written so a different PDF or TXT book can replace it without redesigning the system.

1 How To Use This Handbook

This handbook is designed for a reader who wants more than a notebook that appears to work once. It assumes you want to understand why the system is layered the way it is, what breaks when one layer drifts, and how to tell the difference between a prompt problem, a retrieval problem, and a data-shaping problem. If you only copy the code cells, you will still end up with a working Polymind build. If you read the surrounding system notes, failure analyses, and benchmark guidance, you will also know how to keep that build healthy when you change the book, the model, or the runtime.

There are three productive ways to read the material. The first is the build path. In that path, you move linearly from the Colab setup, through ingestion, chunking, embeddings, retrieval, generation, RL, memory, and finally system assembly. This is the right path if you want to stand up the full artifact set for the first time and see where each persisted file comes from. The second is the debugging path. In that path, you jump straight to chunk inspection, retrieval inspection, reward inspection, the evaluation framework, and the failure playbooks in the appendices. This is the right path when the system is already running but answers feel suspicious, retrieval looks narrow, or the PDF export is hard to trust. The third is the experimentation path. In that path, you build once, rely on `load_from_artifacts()`, and then move through the benchmark fixture, notebook sweeps, ablations, and performance sections to study how the system behaves under different choices.

The guide intentionally repeats a few ideas across different sections. That is not redundancy for its own sake. In real AI systems, the same concept often reappears under different names as you move from data engineering to modeling to evaluation. Chunk size is one example. In the chunking section it is a segmentation decision. In the retrieval section it becomes a recall-precision tradeoff. In the generation section it becomes a prompt budget decision. In the optimization section it becomes a runtime and cost lever. Repetition across those contexts makes the system easier to reason about because it shows how one design choice propagates.

This is also why *Mastery.pdf* is used throughout the book even though the system itself is generic. A concrete reference text makes it possible to show real failure modes rather than abstract warnings. Front matter in *Mastery* creates chapter false positives. Abbreviations such as Dr. and U.S. create sentence-splitting edge cases. Themes like leadership and social intelligence appear across multiple regions of the book, which makes retrieval diversity measurable instead of theoretical. The benchmark fixture later in the handbook uses those same properties so that evaluation questions have a meaningful relationship to the source document.

One more reading note is important: this handbook treats the local repo, the notebook, and the rendered PDF as a single deliverable. The Markdown guide is the canonical narrative source. The `polymind/` package is the canonical logic source. The notebook is the canonical execution walkthrough. If any of the three drift apart, the project becomes harder to debug and harder to trust.

That is why the later validation script checks guide structure, code line lengths, notebook coverage, benchmark fixture richness, and the final rendered page count together. The handbook is not complete just because the prose reads well; it is complete when the prose, code, experiments, and export path all agree on what Polymind V2 actually is.

2 Build Map And Artifact Glossary

The simplest mental model for the project is to think in artifacts rather than functions. At the start there is only a source book, usually `Mastery.pdf`. After ingestion and cleaning, the system has a cleaned text representation and a set of structured sections. After chunking, it has a pool of chunk records with reliable offsets and chapter metadata. After embeddings, it has `embeddings.npy` and a compatible FAISS index. After question answering, it accumulates policy statistics and conversation memory. Each stage becomes smaller and more structured than the one before it. That is why restart safety is feasible: the pipeline is not a single hidden state, but a chain of explicit intermediate contracts.

The core Drive-backed artifacts are worth understanding before you read the code cells. `chunks.json` stores the retrieval substrate the generator ultimately depends on. `embeddings.npy` stores the dense geometry that powers similarity search. `metadata.json` records build context, stage profiles, and enough information to diagnose whether the current runtime is aligned with the saved artifacts. `faiss.index` stores the actual search structure. Finally, `rl_bandit_stats.json` stores the running preference estimates for the answer arms. If any one of these is missing or inconsistent, the resume path becomes less trustworthy. Later appendices provide a fuller contract for each file, but it helps to know their purpose up front.

The public package surface is intentionally narrow. `PolymindConfig` is the single source of truth for paths, model choices, chunking limits, reward weights, epsilon decay, experiment grids, and token budgets. `PolymindSystem` orchestrates the build and exposes the main workflow calls. The individual modules remain important because that is where the implementation lives, but the top-level workflow is designed around the public methods that a notebook or human operator can remember without scanning the whole codebase. This is also why the introspection methods live on the system object rather than only inside helper modules. The System introspection layer is deliberately public because a deep-reading assistant is much easier to trust when chunk shape, retrieval evidence, reward composition, and memory state can be inspected from the same interface that produces answers.

The final glossary item to keep in mind is the distinction between baseline RAG and RL-enhanced selection. Baseline RAG in this project still uses retrieval and grounded prompting. The RL layer adds candidate answer comparison and policy preference on top of that. That distinction matters because later benchmarks are not comparing a grounded system to a hallucination-prone system. They are comparing two grounded systems that differ in how they choose among answer styles. This makes the results more interpretable and keeps the RL layer honest.

If you are reading this guide under time pressure, the minimum effective loop is straightforward. Run the setup. Index the book once. Verify chunk quality with `inspect_chunks()`. Verify retrieval behavior with `inspect_retrieval()` on a few benchmark questions. Run `ask()` with RL enabled. Read

the reward breakdown and memory state. Then use the benchmark cells and appendices to decide whether you want to optimize chunk size, retrieval parameters, model choice, or answer selection policy. That loop captures the heart of the system and matches how a production-minded engineer would actually work with the project.

3 1. Introduction

3.1 Intuition

Most reading assistants fail for a simple reason: they treat a book as a bag of tokens instead of as a long-form structured artifact with chapters, rhetorical flow, recurring themes, and references that develop across hundreds of pages. A single prompt over raw extracted text is not a deep reading system. It is a context overrun waiting to happen. Polymind V2 takes the opposite position. It assumes that the intelligence of the system comes from how it stages work, not just from the size of the generator.

That stance matters even more in Colab. Free or low-cost runtimes give you transient disks, variable GPU access, and real memory ceilings. If the system recomputes embeddings after every disconnect, or if it tries to embed an entire book in one shot, the project becomes brittle. The version in this handbook is designed to survive restarts, degrade gracefully to CPU, and expose the internal state needed to debug real failures instead of hiding them behind notebook magic.

The final deliverable is not just a question answering demo. It is a reading workbench. It can inspect chunk quality, explain retrieval decisions, compare reward signals between answer arms, summarize earlier conversation turns, benchmark baseline RAG against RL-enhanced selection, and surface enough telemetry that a reader can reason about engineering tradeoffs rather than copy cells mechanically.

3.2 Colab-ready code cell

```
from pathlib import Path

from polymind import PolymindConfig

config = PolymindConfig(project_root=Path("/content/drive/MyDrive/polymind_v2"))
print({
    "project_root": str(config.project_root),
    "artifact_dir": str(config.artifact_dir),
    "pipeline": (
        "Book -> Clean -> Structure -> Chunk -> Embed -> Index -> "
        "Retrieve -> Generate -> Score -> Select -> Memory"
    ),
    "default_generator": config.generator_model_name,
    "default_embedder": config.embedding_model_name,
})
```

3.3 Expected output

- A dictionary showing the Drive-backed project root and artifact directory.
- The architecture string exactly as used throughout the guide.
- Default model names that make the design decisions concrete from the start.

3.4 Debugging tips

- If `artifact_dir` points to local ephemeral storage instead of Drive, restart safety is already broken.
- If the config prints a CPU-only path but you expected a GPU runtime, fix the Colab hardware setting before loading the generator.
- If you intend to use a different book later, change the data path, not the architecture or artifact naming scheme.

4 2. System Overview

4.1 Intuition

Polymind V2 is intentionally not a monolith. Each layer narrows uncertainty for the next layer. Ingestion extracts the book and tries to preserve heading signals. Cleaning removes OCR and layout noise. Chunking creates retrieval units with trustworthy offsets. Embeddings convert those units into a searchable geometry. Retrieval proposes evidence. Generation transforms evidence into answer candidates. The RL bandit does not replace evaluation; it provides a lightweight adaptive preference model over answer styles. Memory makes the system conversational without polluting retrieval with blind string concatenation.

This section is also where the first major design tradeoffs become visible. FAISS is chosen over a managed vector database because Colab notebooks need local, zero-ops, single-file persistence more than they need multi-tenant scaling. MiniLM embeddings are chosen because they deliver strong retrieval quality per watt and fit cleanly into CPU fallback paths. A multi-armed bandit is chosen over PPO because the problem is arm selection, not token-level policy optimization. Chunk size and overlap are fixed by a retrieval objective: enough local coherence to anchor semantics, enough overlap to preserve transitions, but not so much duplication that prompts waste context on repeated passages.

Another tradeoff concerns model size. `flan-t5-xl` is substantially better than `flan-t5-base` at conceptual synthesis and long-form explanation, but the larger model only makes sense when loaded with 8-bit quantization and careful cache cleanup. The system therefore separates the retrieval model from the generation model and keeps the retrieval stack lightweight even when generation grows. This is what allows the same project to run on a T4, recover from a CPU-only runtime, and still remain pedagogically consistent.

4.2 Colab-ready code cell

```
architecture = {  
    "ingestion": "PDF/TXT extraction, heading capture, cleaning, chapter detection",  
    "chunking": (  
        "NLTK sentence segmentation, overlap-aware semantic chunks, "  
        "offset tracking"  
    ),  
    "retrieval": (  
        "MiniLM embeddings, FAISS search, HyDE expansion, dedup, "
```



```
    "chapter diversity"
  ),
  "generation": "FLAN-T5 prompts, multi-arm candidate answers, citation verification",
  "selection": "Bandit preference + reward breakdown, single-pass scoring",
  "memory": "Rolling turns + compressed summary + LLM query rewriting",
  "observability": (
    "inspect_chunks, inspect_retrieval, inspect_rewards, "
    "inspect_memory"
  ),
}
architecture
```

4.3 Expected output

- A compact architecture dictionary that mirrors the package module layout.
- A visible reminder that observability is a first-class subsystem, not an afterthought.
- A clean mapping from system behaviors to concrete engineering responsibilities.

4.4 Debugging tips

- If a notebook cell starts duplicating logic that belongs in one of these layers, move it back into the package immediately.
- If retrieval quality is weak, do not reach for a bigger generator first; inspect chunking and embedding behavior.
- If answer quality varies wildly between runs, inspect reward breakdowns and the epsilon schedule before changing prompts.

5 3. Colab Setup

5.1 Intuition

A Colab-first system needs a deterministic setup cell because the runtime is disposable. That means one place to install exact dependencies, one place to mount Drive, one place to seed random number generators, one place to detect the device, and one place to establish logging. Without that discipline, every notebook restart becomes a slightly different environment and debugging turns into folklore.

There is also a practical production reason for the setup cell to be explicit: package versions matter here. `bitsandbytes`, `transformers`, `torch`, `faiss-cpu`, and `pymupdf` interact in ways that can silently fail if versions drift. The guide therefore treats `requirements.txt` as a reproducibility artifact, not a convenience list. The notebook installs from it and then works with the local package code mounted from Drive.

Finally, setup is where we make resource limits visible. The GPU may exist, but it may not have enough free memory for the large generator. The setup cell therefore prints the device, configures logging, and seeds everything early so that performance experiments later in the guide are at least directionally comparable.

5.2 Colab-ready code cell

```
!pip install -q -r /content/drive/MyDrive/polymind_v2/requirements.txt

from google.colab import drive
drive.mount("/content/drive")

import logging
import random
from pathlib import Path

import numpy as np
import torch

random.seed(42)
np.random.seed(42)
torch.manual_seed(42)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(42)
```

```
PROJECT_ROOT = Path("/content/drive/MyDrive/polymind_v2")
DATA_DIR = PROJECT_ROOT / "data"
ARTIFACT_DIR = PROJECT_ROOT / "artifacts"
PROJECT_ROOT.mkdir(parents=True, exist_ok=True)
DATA_DIR.mkdir(parents=True, exist_ok=True)
ARTIFACT_DIR.mkdir(parents=True, exist_ok=True)

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s | %(levelname)s | %(name)s | %(message)s",
)
device_name = "cuda" if torch.cuda.is_available() else "cpu"
print({"device": device_name, "project_root": str(PROJECT_ROOT)})
```

5.3 Expected output

- Successful dependency installation from the pinned requirements file.
- A mounted Drive path at /content/drive.
- A device printout showing either cuda or cpu.
- Logging configured at INFO level for the rest of the notebook session.

5.4 Debugging tips

- If bitsandbytes fails to import on CPU, that is acceptable as long as the generator fallback path uses flan-t5-base.
- If Drive mounting fails, do not continue. Artifact persistence is central to the design.
- If package imports later resolve to stale notebook state, use Runtime -> Restart session and rerun this setup cell cleanly.

6 4. Project Structure

6.1 Intuition

A notebook-only architecture is the fastest way to lose control of a real AI system. Cells get rerun out of order, hidden state accumulates, and debugging becomes a memory game. Polymind V2 uses a hybrid layout instead: the notebook orchestrates, while the `polymind/` package owns the logic. This keeps the system teachable in Colab without sacrificing modularity.

The most important structural choice is that persistence and observability live in the package rather than in the notebook. That means `load_from_artifacts()`, `inspect_chunks()`, `inspect_retrieval()`, `inspect_rewards()`, and `inspect_memory()` are all callable from the top-level `PolymindSystem`. The notebook therefore becomes a transparent control surface rather than a second implementation.

This structure also improves PDF export. A long handbook full of code snippets is only trustworthy if those snippets correspond to real files. Because the modules exist in the repo, the guide can point at stable interfaces and the notebook can import them step by step without hiding logic in Markdown exposition.

6.2 Colab-ready code cell

```
project_map = {
    "polymind/config.py": "Typed config, dataclasses, artifact paths, stage profiles",
    "polymind/ingestion.py": (
        "PDF/TXT loading, cleaning, font-aware heading extraction, "
        "chapter detection"
    ),
    "polymind/chunking.py": "Sentence splitting, chunk construction, chunk inspection",
    "polymind/embeddings.py": "Batched MiniLM embeddings, OOM fallback, profiling",
    "polymind/retrieval.py": (
        "FAISS build/load, HyDE, dedup, chapter diversity, "
        "retrieval inspection"
    ),
    "polymind/generation.py": (
        "Quantized FLAN loading, prompt building, candidate generation, "
        "citation checks"
    ),
    "polymind/rl.py": "Bandit policy, reward computation, reward inspection",
    "polymind/memory.py": (
```

```
    "Memory summary compression, query rewriting, memory inspection"
),
    "polymind/system.py": "PolymindSystem orchestration, persistence, public API",
    "notebooks/polymind_v2_colab.ipynb": "Thin orchestration notebook",
    "tests/": "Logic regression tests and API smoke checks",
}
project__map
```

6.3 Expected output

- A dictionary that explains what each file owns.
- A clear separation between package logic and notebook orchestration.
- A mental model for where to debug specific failures later in the guide.

6.4 Debugging tips

- If a function in the notebook feels reusable, move it into the package before continuing.
- If two modules need the same constant, put it into PolymindConfig instead of copying it.
- If an inspection method only exists in the notebook and not in PolymindSystem, the architecture is drifting.

7 5. Book Ingestion

7.1 Intuition

Book ingestion is the first place where “tutorial code” usually collapses under real documents. PDFs can carry font metadata, page layout quirks, and front-matter formatting that simple text extraction destroys. TXT files are much simpler, but then you lose typography-driven heading cues. Polymind V2 supports both, but it treats PDFs as privileged sources because they let us exploit font size when chapter headings are visually distinct.

For *Mastery.pdf*, this matters. The book includes front matter, section changes, and headings that are much easier to detect when font information is preserved. The system therefore attempts PyMuPDF extraction with `page.get_text("dict")` to gather large-font lines, falls back to plain text extraction when needed, and merges the heading candidates into regex-based chapter detection. This hybrid approach is more resilient than regex alone.

The ingestion stage is also where restart safety begins. The raw text can be re-extracted, but downstream work should not rerun once the clean sections and artifacts exist. That is why the system separates `ingest_book()` from `index_book()`: ingestion gathers truth from the source document, while indexing decides whether to recompute or reload persisted artifacts.

7.2 Colab-ready code cell

```
from polymind import PolymindConfig
from polymind.ingestion import ingest_book

config = PolymindConfig(project_root=PROJECT_ROOT)
ingestion = ingest_book(DATA_DIR / "Mastery.pdf", config=config)

print(ingestion["metadata"])
print("font headings:", len(ingestion["font_headings"]))
print("chapters:", ingestion["chapters"][:5])
if ingestion["sections"]:
    first_title = ingestion["sections"][0].chapter_title
else:
    first_title = "none"
print("first section title:", first_title)
```

7.3 Expected output

- Metadata summarizing character counts and detected section counts.
- A count of font-derived heading candidates.
- A preview of detected chapter boundaries.
- The first resolved section title, which is a fast sanity check for front-matter handling.

7.4 Debugging tips

- If font headings is zero on a PDF, confirm that pymupdf installed correctly and that the document is not image-only.
- If chapter detection captures front matter as chapters, inspect the first few heading candidates before changing regexes.
- If a PDF extracts almost no text, test the document manually; some scans require OCR before this pipeline can help.

8 6. Text Cleaning

8.1 Intuition

Raw extraction is noisy in ways that directly hurt retrieval. Ligatures such as fi and fl, repeated whitespace, long runs of blank lines, and layout dashes will all distort sentence segmentation and embedding space if left untouched. Cleaning is therefore not cosmetic. It is normalization in service of later geometry.

The trick is to clean aggressively enough to stabilize downstream stages without erasing real semantic structure. Polymind V2 normalizes whitespace, replaces common ligatures, harmonizes em dashes, and preserves paragraph boundaries. It does not perform heavy semantic rewriting or stopword removal at this stage because the original surface form is still valuable for citations, debugging, and offset calculations.

This is also the first point where observability pays off. If retrieval becomes erratic, inspect a cleaned sample before assuming the embedding model is weak. Many poor retrieval runs are caused by upstream text corruption, not by similarity search itself.

8.2 Colab-ready code cell

```
from polymind.ingestion import clean_text, read_book_text

raw_text = read_book_text(DATA_DIR / "Mastery.pdf")
cleaned_text = clean_text(raw_text)

print("raw chars:", len(raw_text))
print("clean chars:", len(cleaned_text))
print(cleaned_text[:1200])
```

8.3 Expected output

- A small difference between raw and cleaned character counts.
- Cleaner early pages with normalized whitespace and replaced ligatures.
- Enough visible text to catch formatting noise before chapter detection.

8.4 Debugging tips

- If cleaned text gets much shorter, the cleaning step is too destructive.
- If ligatures or page junk remain common, extend the replacement table in `clean_text()`.
- If the first 1,200 characters still look like a table of contents, that is not a cleaning bug; it is an ingestion boundary issue.

9 7. Chapter Detection

9.1 Intuition

Chapter detection is a surprisingly high-leverage part of deep reading. Good chapter boundaries improve chunk labels, retrieval diversity, summary coverage, and citation quality. Bad chapter boundaries produce a cascade of subtle failures: chunks receive misleading chapter IDs, retrieval collapses into the wrong region of the book, and chapter-level summaries become incoherent because the evidence pool is mislabeled.

Polymind V2 uses three signals together. First, regex patterns catch conventional headings. Second, large-font lines from the PDF contribute visual candidates. Third, a character-window deduplication rule collapses near-duplicate matches that arise when multiple regexes or font spans point at the same heading with slightly different offsets. This directly fixes the phantom-duplicate behavior that often corrupts section construction.

The system deliberately keeps the detection logic explainable. That is a design choice. A more opaque classifier might improve recall in edge cases, but the chapter detector here is part of the debugging story. When something looks wrong, the reader should be able to inspect candidate spans, reason about the dedup window, and adjust heuristics with confidence.

9.2 Colab-ready code cell

```
from polymind.ingestion import (
    build_book_sections,
    clean_text,
    detect_chapters,
    extract_font_headings,
)

cleaned = clean_text(read_book_text(DATA_DIR / "Mastery.pdf"))
font_headings = extract_font_headings(DATA_DIR / "Mastery.pdf")
chapters = detect_chapters(cleaned, font_headings=font_headings)
sections = build_book_sections(cleaned, chapters, source_path=DATA_DIR / "Mastery.pdf")

print("detected chapters:", len(chapters))
for chapter in chapters[:8]:
    print(chapter)
print("first three sections:", [section.chapter_title for section in sections[:3]])
```

9.3 Expected output

- A plausible chapter count rather than dozens of repeated near-matches.
- Early chapter tuples with clean titles and ascending offsets.
- The first section titles matching actual book structure instead of duplicated headings.

9.4 Debugging tips

- If the same chapter appears twice with offsets only a few characters apart, widen the dedup window or inspect the font candidates.
- If all-uppercase lines from the front matter are getting through, add a content heuristic before changing the main patterns.
- If sections look correct but chapter titles are ugly, normalize titles rather than weakening detection.

10 8. Semantic Chunking

10.1 Intuition

Chunking determines what retrieval is even allowed to see. If chunks are too small, the retriever returns fragments with no explanatory power. If they are too large, retrieval loses precision and generation gets crowded prompts. Polymind V2 uses target-length semantic chunks with sentence-aware overlap and, critically, correct offset tracking. The earlier `running_char` shortcut is removed entirely because it could not maintain trustworthy source spans across overlapping buffers.

The implementation scans the original section text sentence by sentence and records the true local start and end positions of each sentence. Chunks are then defined by the first and last sentence in the buffer. This is slower than a naive counter, but it is correct, and correctness matters because citations and source tracing are only meaningful when offsets correspond to real positions in the book.

Chunking is also the first stage with explicit introspection. `inspect_chunks()` exposes word counts, sentence counts, overlap ratios, offset ranges, and warning flags such as `under_target`, `over_target`, and `low_sentence_count`. This turns chunking into a measurable subsystem. If retrieval later behaves strangely, you can test whether the evidence pool itself is malformed instead of guessing.

10.2 Colab-ready code cell

```
from polymind.chunking import inspect_chunks, semantic_chunk_book

chunks = semantic_chunk_book(sections, config)
chunk_rows = inspect_chunks(chunks, config, limit=10)

print("chunk_count:", len(chunks))
for row in chunk_rows:
    print(row)
```

10.3 Expected output

- A chunk count in the hundreds for a full book like *Mastery*.
- Inspection rows that show stable word counts near the configured target.
- Monotonic offset ranges and moderate overlap ratios instead of wild jumps.

10.4 Debugging tips

- If many chunks are under `__target`, your chapter boundaries or sentence splitter may be broken.
- If offsets overlap excessively or move backward, stop and fix chunking before building embeddings.
- If chunk previews show repeated text almost verbatim, reduce overlap or improve dedup in retrieval.

11 9. Chunk Quality Inspection

11.1 Intuition

Inspection is the hinge that turns this project from a tutorial into an engineering handbook. Once chunking exists, the next temptation is to assume it works because the code runs. That is exactly how broken offsets, degenerate chunk sizes, and repeated passages leak into later stages. `inspect_chunks()` exists to make the chunk pool visible before embeddings hide errors behind similarity scores.

In practice, the most useful chunk inspection signals are boring and mechanical: word count, sentence count, overlap ratio, offset range, and a preview. Those are enough to spot the common failure modes. A spike in `under_target` chunks often means chapter detection produced tiny sections. A high overlap ratio across long stretches often indicates the chunker is buffering too aggressively. Invalid offsets mean citations cannot be trusted, which means evaluation downstream is contaminated even if answers sound good.

For *Mastery.pdf*, inspection is especially important around the early pages. Front matter, section headers, and typography changes can create short, suspicious chunks before the main body settles down. The guide uses those cases as diagnostic examples because they resemble the kinds of partial failures that appear in real books, not just synthetic test strings.

11.2 Colab-ready code cell

```
import pandas as pd

chunk_report = pd.DataFrame([row.__dict__ for row in system.inspect_chunks(limit=20)])
display(chunk_report)

problem_chunks = chunk_report[chunk_report["warnings"].map(bool)]
problem_cols = [
    "chunk_id",
    "chapter_title",
    "word_count",
    "warnings",
    "preview",
]
display(problem_chunks[problem_cols])
```

11.3 Expected output

- A dataframe of chunk inspection rows with word counts, offsets, and warnings.
- A filtered problem list that immediately highlights suspicious chunks.
- Enough context in the previews to decide whether the issue is structural or merely cosmetic.

11.4 Debugging tips

- If every early chunk is suspicious, inspect chapter detection before retuning chunk size.
- If only the last chunk of every chapter is too short, consider whether the fallback behavior is acceptable before changing the algorithm.
- If chunk previews show broken abbreviations or chopped sentences, re-check the sentence splitter rather than compensating with larger chunks.

12 10. Embeddings

12.1 Intuition

Embeddings turn the chunk pool into a searchable vector space, but in Colab the real engineering problem is not “how do embeddings work?” It is “how do we embed a whole book safely and reproducibly?” The answer is batching, explicit progress tracking, and a fallback path when the GPU runs out of memory. A single monolithic `encode()` call can fail silently or unpredictably on free-tier runtimes. A production-friendly loop makes the work inspectable and restartable.

MiniLM is the default embedding model because it delivers a strong quality-to-cost ratio for retrieval. The goal is not to maximize semantic richness at any price; it is to make retrieval stable enough that the generator receives grounded evidence even on CPU. That is why the guide keeps the embedding model lightweight while allowing the generator to scale up when the runtime permits.

Observability continues here through stage profiling. Embedding time and peak memory are worth measuring because they frame later optimization choices. If embeddings dominate runtime, chunk size experiments have a direct operational cost. If embedding batches frequently trigger CPU fallback, the runtime may be too constrained for the current settings.

12.2 Colab-ready code cell

```
from polymind.embeddings import load_embedding_model

embed_model = load_embedding_model(config.embedding_model_name)
embedding_result = system.index_book(DATA_DIR / "Mastery.pdf", force_recompute=True)
print(embedding_result["profiles"])
print("embedding matrix shape:", system.chunk_embeddings.shape)
```

12.3 Expected output

- A profile list showing ingestion, chunking, embedding, and index timings.
- An embedding matrix shape consistent with the number of chunks and the embedding dimension.
- A successful run that writes `embeddings.npy` to the artifact directory.

12.4 Debugging tips

- If embedding fails with a CUDA OOM, confirm that the fallback path moved the model to CPU and reduced batch size.
- If the matrix shape is (0, 384) or similar, chunk generation failed upstream.
- If embedding time changes wildly between runs on the same runtime, inspect whether the GPU was preloaded by other notebooks.

13 11. FAISS Index

13.1 Intuition

FAISS is the right index here because it is local, fast, portable, and friendly to a single-book workload. A managed vector database would add network hops, credentials, and operational complexity without helping the main objective of this guide. The index must be buildable inside Colab, serializable to Drive, and reloadable after runtime resets. That is the operational profile FAISS fits well.

The subtle part is persistence across heterogeneous runtimes. A notebook may build the index under one device context and reload it under another. The guide therefore normalizes the FAISS artifact by reserializing it after load. That small step avoids a class of CPU/GPU mismatch headaches that are easy to dismiss as environment flukes but painful to debug mid-session.

The index stage is also where restart safety becomes tangible. If `faiss.index`, `chunks.json`, and `embeddings.npy` exist, there is no reason to reprocess the book. That is why `PolymindSystem.load_from_artifacts()` is treated as a first-class path rather than a convenience helper.

13.2 Colab-ready code cell

```
loaded = system.load_from_artifacts()
print("loaded_from_artifacts:", loaded)
print("index_ready:", system.index is not None)
print("chunks_path:", config.chunks_path())
print("index_path:", config.index_path())
```

13.3 Expected output

- `loaded_from_artifacts:` True after the first successful indexing pass.
- A non-null index object ready for retrieval.
- Artifact paths pointing into the Drive-backed artifact directory.

13.4 Debugging tips

- If `loaded_from_artifacts` is false after a prior run, inspect whether the notebook wrote artifacts to ephemeral storage.
- If the index loads but retrieval crashes, confirm that `chunks.json` and `embeddings.npy` correspond to the same build.
- If FAISS is unavailable locally during testing, the package falls back to a numpy index; that is fine for tests but not for performance benchmarking.

14 12. Retrieval System

14.1 Intuition

Retrieval is the core intelligence filter of the system. Everything after it depends on whether the evidence set is relevant, diverse, and compact enough to fit into the generator's working context. Polymind V2 improves retrieval along four dimensions at once: query expansion with HyDE, top-k search over dense embeddings, cosine deduplication to remove overlapping chunks, and chapter diversity to prevent the result set from collapsing into one narrow region of the book.

This section is where the introspection layer becomes indispensable. `inspect__retrieval()` shows the original question, the rewritten question, the hypothetical answer used for expansion, the raw top-k hits, the chunks removed as near-duplicates, and any diversity decisions. That is the shortest path from “the answer felt weak” to “retrieval over-concentrated on chapter 2 and dropped a good chapter-7 chunk.”

The retrieval design tradeoff is deliberate. Managed ranking models or hybrid sparse-dense stacks can outperform this setup in larger systems, but they also complicate reproducibility and pedagogy. For one-book deep reading, a carefully instrumented dense retriever with diversity rules is enough to teach the right engineering habits while producing genuinely useful behavior.

14.2 Colab-ready code cell

```
retrieval_debug = system.inspect__retrieval(
    "How does Greene connect social intelligence and strategic patience?",
    top_k=8,
)

inspection = retrieval_debug["inspection"]
print("rewritten_query:", inspection.rewritten_query)
print("hyde_query:", inspection.hyde_query)
print("dedup_events:", inspection.dedup_events)
print("diversity_events:", inspection.diversity_events)
print("selected_results:", retrieval_debug["results"])
```

14.3 Expected output

- A rewritten query that is more self-contained than the original follow-up.
- A HyDE-style hypothetical answer paragraph or an empty string if disabled.
- Visible dedup and diversity events when overlapping or same-chapter hits are removed.
- A final selected result list with similarity scores and chapter labels.

14.4 Debugging tips

- If `hyde_query` is empty on a GPU runtime, inspect the generator interface passed into retrieval.
- If all selected results come from one chapter, raise `top_k` first and only then consider relaxing diversity limits.
- If dedup removes too much context, inspect chunk overlap settings before weakening the cosine threshold.

15 13. RAG Pipeline

15.1 Intuition

RAG is not a model. It is a contract between retrieval and generation. Retrieval promises to provide evidence that is relevant and compact. Generation promises to stay inside that evidence when answering. The usefulness of the final answer depends on both sides honoring the contract. This is why the prompt builder includes explicit evidence formatting and citations while the retriever tries to eliminate duplicate or narrow evidence beforehand.

The system prompt in Polymind V2 is intentionally simple. It names the assistant, instructs it to stay grounded, and tells it how to cite chunks. The sophistication lives more in evidence curation than in ornate prompt engineering. That is a design lesson worth emphasizing: for a book QA system, better retrieval often beats clever prompt text.

This section also introduces memory-aware retrieval and generation boundaries. The memory summary is provided as context, but the rewritten question remains the actual retrieval key. That keeps memory useful without letting conversation history swamp the embedding space. It is a small implementation detail with large quality consequences.

15.2 Colab-ready code cell

```
from polymind.generation import build_grounded_prompt

retrieval_debug = system.inspect_retrieval("What does Greene mean by apprenticeship?")
prompt = build_grounded_prompt(
    question=retrieval_debug["inspection"].rewritten_query,
    retrieved_results=retrieval_debug["results"],
    instruction=(
        "Answer directly, cite supporting chunks, and stay grounded "
        "in the book."
    ),
    memory_context=system.memory.build_memory_summary(),
)
print(prompt[:2500])
```

15.3 Expected output

- A prompt that clearly separates question, memory, and retrieved context.
- Chunk blocks labeled with chapter and chunk IDs.
- An instruction style that is readable and minimal rather than excessively prompt-heavy.

15.4 Debugging tips

- If the prompt is already too long before generation, reduce duplicate evidence or lower top_k.
- If the answer later hallucinates citations, inspect whether the prompt includes chunk IDs consistently.
- If memory context dominates the prompt, the summarization policy is too verbose.

16 14. Generator Model

16.1 Intuition

The generator is where users feel the system, but it should not dictate the rest of the design. `flan-t5-xl` is the default because it is materially better than `flan-t5-base` at explanation, synthesis, and multi-paragraph reasoning. On a T4, the difference between feasible and impossible is 8-bit quantization. That is why the loading path uses `BitsAndBytesConfig`, safe device mapping, and a CPU fallback model instead of assuming the large model will always fit.

This design also embodies a model-size versus latency tradeoff. Larger models improve answer quality, but they cost more memory and time per candidate. Because Polymind V2 generates multiple candidate answers per question, model inefficiency compounds quickly. The solution is not to abandon multi-arm generation; it is to load the generator safely, tune task-specific token budgets, and clear GPU cache between candidate generations.

Benchmarking later in the guide will compare `flan-t5-base` and `flan-t5-xl` directly. The aim is not to crown a universally best model. It is to show how answer quality, latency, and Colab reliability move together so that model choice becomes an engineering decision rather than a status signal.

16.2 Colab-ready code cell

```
from polymind.generation import load_generator

generator = load_generator(config)
print({
    "model_name": generator.model_name,
    "device": generator.device,
    "use_8bit": config.use_8bit_quantization,
    "qa_tokens": config.token_budget("qa"),
    "deep_tokens": config.token_budget("deep"),
})
```

16.3 Expected output

- A generator bundle showing the resolved model name and device.

- `flan-t5-xl` on GPU when resources allow, and the fallback model on CPU-only runtimes.
- Token budgets that differ by task type instead of using one global limit.

16.4 Debugging tips

- If loading `flan-t5-xl` crashes, inspect whether quantization was actually applied and whether the runtime has enough free VRAM.
- If the model loads on CPU unexpectedly, verify the Colab accelerator setting and `torch.cuda.is_available()`.
- If deep summaries truncate mid-thought, increase the task-specific token budget rather than the global default.

17 15. Answer Generation

17.1 Intuition

Polymind V2 generates multiple grounded answers because there is no single best explanation style for every question. Some questions reward directness, others reward pedagogy, and still others need synthesis across distant passages. The system expresses these styles as answer arms with distinct temperatures and instructions. The generator produces all of them, then the RL layer evaluates which one earned the best grounded reward.

This is also where citation verification matters. Large language models can invent chunk IDs even when the prompt is explicit. The system therefore strips invalid citations after generation and only preserves those that correspond to the actual retrieved set. That makes the answer safer to trust and makes evaluation meaningful later.

Operationally, answer generation is also where Colab memory pressure spikes. Three candidate answers back-to-back can exhaust a T4 if the cache is not cleared. The guide therefore treats `torch.cuda.empty_cache()` as part of the normal generation loop, not as an emergency workaround.

17.2 Colab-ready code cell

```
from polymind.generation import generate_answer_candidates

retrieval_debug = system.inspect_retrieval(
    "Why does Greene emphasize social intelligence?"
)
candidates = generate_answer_candidates(
    question=retrieval_debug["inspection"].rewritten_query,
    retrieved_results=retrieval_debug["results"],
    generator=system.generator or generator,
    config=config,
    task="qa",
    memory_context=system.memory.build_memory_summary(),
)

for candidate in candidates:
    print(candidate.arm_name, candidate.temperature, candidate.citations)
```

```
print(candidate.answer[:400])  
print(" " * 80)
```

17.3 Expected output

- Three candidate answers with visibly different explanation styles.
- Temperatures aligned with the configured answer arms rather than defaulting silently to one value.
- Citation lists containing only verified references from the retrieved set.

17.4 Debugging tips

- If all three answers sound identical, inspect whether arm temperatures and instructions are being passed through correctly.
- If citations are always empty, check whether the prompt formatting still exposes chunk IDs clearly.
- If generation OOMs on the second or third arm, inspect cache cleanup and reduce token budgets before reducing model size.

18 16. Reward Function

18.1 Intuition

The reward function is where Polymind V2 makes its strongest quality correction. A lexical hallucination penalty punishes paraphrase, which is exactly what a good explanatory model often does. The replacement is an embedding-based divergence score: compare the answer to the retrieved evidence and optionally against a held-out non-selected chunk to detect drift without requiring surface-form copying. This preserves expressive answers while still discouraging unsupported wander.

Reward is deliberately multi-component. Grounding similarity measures how close the answer embedding is to the mean embedding of the retrieved evidence. Normalized keyword overlap rewards coverage of salient terms without giving a free pass to very long answers. Citation validity rewards grounded references. Divergence penalty subtracts score when the answer aligns more strongly with a held-out chunk than with the evidence that was actually retrieved.

Inspection is again central. `inspect_rewards()` exposes the full reward breakdown per candidate and records the bandit's preferred arm and tie-break behavior. This is the fastest way to understand why the system chose a teacher-style answer over a concise one, or why the synthesizer underperformed on a narrow factual question.

18.2 Colab-ready code cell

```
answer = system.ask("How does Greene connect mastery to leadership?", use_rl=True)

print("selected_arm:", answer["selected_arm"])
for reward in answer["candidate_rewards"]:
    print(reward)

print(system.inspect_rewards())
```

18.3 Expected output

- A selected arm name corresponding to one of the configured answer strategies.
- Reward breakdowns with grounding, overlap, citation, and divergence components.

- A top-level reward inspection object showing epsilon and tie-break context.

18.4 Debugging tips

- If every arm receives almost the same reward, the reward model may be too weak or the retrieved evidence may be too homogeneous.
- If longer answers always win, inspect the overlap normalization baseline.
- If divergence is always zero, confirm that a held-out chunk is actually being supplied from the raw retrieval pool.

19 17. RL Bandit Layer

19.1 Intuition

The reinforcement learning layer in Polymind V2 is intentionally small and well-scoped. The system is not trying to learn token-by-token generation policies. It is trying to learn which answer style tends to win for this workload. That makes the problem a natural fit for a multi-armed bandit. Each answer arm is an action. Each reward breakdown is a scalar outcome. The policy then balances exploration and exploitation with an epsilon schedule that decays over time.

This choice matters pedagogically and operationally. PPO or other policy-gradient methods would be more complex, more fragile in a notebook setting, and less aligned with the actual decision being made. The bandit layer is not a compromise because the project is small. It is the correct abstraction for answer-arm selection. It also persists cleanly as a JSON stats file, which makes it a natural companion to the rest of the Drive-backed artifacts.

Introspection makes the bandit legible. `inspect_rewards()` shows which arm the policy preferred, which arm actually won, how close the reward scores were, and whether tie-breaking nudged the selection. That is how the guide turns RL from mysterious branding into a debug-friendly engineering layer.

19.2 Colab-ready code cell

```
from pandas import DataFrame

__ = system.ask("Why does Greene value strategic patience?", use_rl=True)

reward_inspection = system.inspect_rewards()
reward_table = DataFrame([row.__dict__ for row in reward_inspection.reward_breakdowns])
display(reward_table)

print({
    "preferred_arm": reward_inspection.preferred_arm,
    "selected_arm": reward_inspection.selected_arm,
    "epsilon": reward_inspection.epsilon,
    "tie_margin": reward_inspection.tie_margin,
})
print(system.policy.learning_curve())
```

19.3 Expected output

- A reward table with one row per answer arm.
- Policy metadata showing preferred versus selected arm.
- A lightweight learning-curve view with pulls, wins, and average reward per arm.

19.4 Debugging tips

- If epsilon never drops, inspect whether pulls are being recorded for every evaluated arm.
- If one arm wins almost always from the start, test whether the reward function is too correlated with answer length or citation style.
- If the bandit policy looks random after many interactions, the reward signal is probably too weak or too noisy.

20 18. Memory System

20.1 Intuition

Conversation memory is where many reading assistants quietly sabotage retrieval. The naive move is to append the previous answer to the next question. That bloats the retrieval query and drags unrelated language into embedding space. Polymind V2 instead uses two memory mechanisms with clear roles: a rolling window of recent turns for immediate conversational continuity, and a compressed summary for older turns that would otherwise be dropped.

When recent turns overflow, the system calls `__compress__to__summary()` and asks the generator to distill the dropped conversation into a factual memory summary. This fixes the earlier architectural gap where memory claimed to have a summary field but never actually wrote to it. Follow-up question rewriting is then done by the generator as a small self-contained task, not by concatenating raw history into the query string. The rewritten question becomes the retrieval key, while the memory summary remains supportive context for prompting.

This section is another point where observability matters. `inspect__memory()` exposes the summary text, recent turns, compression event count, and rewrite context. That makes it possible to debug questions like “why did the system suddenly start retrieving leadership passages?” by checking whether memory compression or rewriting introduced a thematic bias.

20.2 Colab-ready code cell

```
__ = system.ask("What is Greene's apprenticeship model?")
__ = system.ask("How does that connect to leadership?")
memory__debug = system.inspect__memory()

print("summary:", memory__debug.summary)
print("compression_events:", memory__debug.compression_events)
for turn in memory__debug.recent_turns:
    print(turn)
print("rewrite_context:", memory__debug.rewrite_context[:800])
```


20.3 Expected output

- Recent turns with original and rewritten questions.
- A summary string once enough turns have accumulated to trigger compression.
- A non-empty rewrite context that explains what the follow-up rewriter saw.

20.4 Debugging tips

- If rewritten follow-ups become too verbose, shorten the rewrite prompt rather than disabling memory entirely.
- If the summary starts hallucinating claims not supported by prior turns, reduce the memory summary token budget and inspect the prompt.
- If retrieval degrades after several turns, compare `inspect_memory()` with `inspect_retrieval()` to see whether rewriting or evidence selection drifted first.

21 19. System Assembly

21.1 Intuition

By the time all subsystems exist, the biggest remaining risk is orchestration drift. A notebook can easily end up calling low-level functions in slightly different ways than the packaged system, creating two behaviors that look similar but are hard to compare. PolymindSystem exists to prevent that. It centralizes indexing, artifact loading, retrieval, generation, RL selection, memory updates, and introspection behind one public interface.

A good system object is not merely convenient. It becomes the stable control surface for experiments, benchmarks, and future extensions. The notebook can call `ask()` and get back the answer, rewritten query, selected arm, reward breakdowns, verified citations, and profiles in one consistent payload. That consistency is what makes later benchmarking credible, because every comparison uses the same top-level call path.

This is also where the guide becomes production-oriented in tone. Once the interface is stable, the question changes from “can the code work?” to “can we reason about it, reload it, inspect it, and extend it safely?” That is the difference between an implementation tutorial and a systems handbook.

21.2 Colab-ready code cell

```
from polymind import PolymindSystem

system = PolymindSystem(config=config)
system.index_book(DATA_DIR / "Mastery.pdf")

response = system.ask("How does Greene define mastery?")
print(response["rewritten_question"])
print(response["selected_arm"])
print(response["verified_citations"])
print(response["answer"])
```

21.3 Expected output

- A top-level response containing rewritten query, selected arm, citations, and the final answer.

- A stable public API that feels like a system object rather than a loose collection of helpers.
- Enough metadata in the response to support later debugging and benchmarking.

21.4 Debugging tips

- If the system works when called module-by-module but not through PolymindSystem, inspect the assembly order and lazy model loading.
- If `ask()` does not surface the rewritten question, fix the response payload before running retrieval experiments.
- If artifact loading and indexing disagree about config values, persist config metadata and compare it on load.

22 20. Persistence & Reloading

22.1 Intuition

Persistence is not a convenience feature in Colab. It is the difference between a viable project and a frustrating one. A deep reading pipeline has several expensive stages: extraction, chunking, embedding, indexing, and policy accumulation. Repeating them after every runtime reset is wasteful and makes experimentation slower than it needs to be. Polymind V2 treats Drive persistence as part of the architecture, not as something bolted on afterward.

The persisted artifact set is intentionally small and explicit: `chunks.json`, `embeddings.npy`, `metadata.json`, `faiss.index`, and `rl_bandit_stats.json`. That makes resume logic easy to audit. The system checks for all required files and calls `load_from_artifacts()` when they exist. Once loaded, the notebook can go straight to question answering, experiments, or summaries.

This section also illustrates a broader production principle: expensive transformations should become stable artifacts with clear versioning boundaries. That makes systems debuggable, rerunnable, and extensible. When later sections compare retrieval strategies or benchmark generators, they can reuse the same persisted chunk and vector substrate rather than recompiling the world each time.

22.2 Colab-ready code cell

```
artifact_status = {
    "chunks": config.chunks_path().exists(),
    "embeddings": config.embeddings_path().exists(),
    "metadata": config.metadata_path().exists(),
    "faiss": config.index_path().exists(),
    "rl_stats": config.rl_stats_path().exists(),
}
print(artifact_status)

loaded = system.load_from_artifacts()
print("loaded:", loaded)
```

22.3 Expected output

- A dictionary of boolean artifact checks.
- loaded: True once the system has already been indexed at least once.
- A resume path that skips recomputation and leaves the system ready to answer questions.

22.4 Debugging tips

- If one artifact is missing, decide whether partial recovery is safe before silently rebuilding everything.
- If reloading works but answers change unexpectedly, compare the config saved in metadata.json with the current config.
- If RL stats vanish across sessions, verify that the policy is being persisted after each question or experiment step.

23 21. Evaluation Framework

23.1 Intuition

Evaluation is where Polymind V2 becomes a system design handbook rather than a build log. There are three complementary goals here. First, measure whether retrieval returns the right evidence. Second, measure whether generation stays grounded and useful. Third, measure whether the RL layer improves answer selection over a fixed baseline. Those goals require experiments, benchmarks, and failure analysis rather than anecdotal inspection.

Experimentation framework. The guide treats experiments as parameterized notebook studies over stable package APIs. Sweep `top_k` to see how evidence breadth affects reward and citation quality. Sweep chunk target size to see how retrieval specificity trades off against context completeness. Compare arm temperature against reward to understand when the synthesizer helps and when it drifts. Plot the bandit learning curve to see whether policy preference converges or remains noisy.

Benchmarking. The minimum benchmark set compares baseline RAG versus RL-enhanced answer selection, `flan-t5-base` versus `flan-t5-xl`, and retrieval before versus after the HyDE plus dedup plus diversity upgrades. Use a small curated set of book questions and report retrieval chapter correctness, verified citation count, average reward, answer length, and latency. For Mastery, good benchmark prompts include apprenticeship, social intelligence, leadership, strategic patience, and cross-chapter synthesis questions because they exercise different retrieval patterns.

Failure modes and recovery. Three failures deserve dedicated diagnosis. Bad chunking usually shows up as many suspicious chunks, repeated previews, or invalid offsets; inspect chunks first, then revisit chapter detection and sentence splitting. Weak retrieval shows up as low-diversity evidence, irrelevant chapters, or near-duplicate passages; inspect rewritten queries, raw top-k hits, dedup events, and chapter counts before changing the generator. RL misbehavior shows up as persistent arm randomness, reward collapse, or one arm winning for the wrong reasons; inspect reward breakdowns, epsilon decay, and normalization before blaming exploration. The point is to trace failure through the introspection tools instead of tuning blindly.

23.2 Colab-ready code cell

```

import pandas as pd
import matplotlib.pyplot as plt

questions = [
    "What is Greene's apprenticeship model?",
    "Why does social intelligence matter?",
    "How does mastery relate to leadership?",
]

rows = []
for top_k in config.experiment_grid["top_k"]:
    for question in questions:
        result = system.ask(question, top_k=top_k, use_rl=True)
        rows.append(
            {
                "question": question,
                "top_k": top_k,
                "selected_arm": result["selected_arm"],
                "max_reward": max(
                    (row["total_reward"] for row in result["candidate_rewards"]),
                    default=0.0,
                ),
                "citations": len(result["verified_citations"]),
                "answer_length": len(result["answer"].split()),
            }
        )

experiment_df = pd.DataFrame(rows)
display(experiment_df)

mean_rewards = experiment_df.groupby("top_k")["max_reward"].mean()
mean_rewards.plot(marker="o", title="Average reward by top_k")
plt.show()

benchmark_rows = []
for use_rl in [False, True]:
    result = system.ask("How does Greene define strategic patience?", use_rl=use_rl)
    benchmark_rows.append(
        {
            "mode": "rl" if use_rl else "baseline",
            "selected_arm": result["selected_arm"],
            "reward": max(
                (row["total_reward"] for row in result["candidate_rewards"]),
                default=0.0,
            )
        }
    )

```

```
        ),
        "citations": len(result["verified_citations"]),
        "answer_length": len(result["answer"].split()),
    }
)

benchmark_df = pd.DataFrame(benchmark_rows)
display(benchmark_df)
print(system.inspect_retrieval())
print(system.inspect_rewards())
print(system.inspect_memory())
```

23.3 Expected output

- An experiment dataframe that supports top-k comparison across multiple questions.
- A simple reward curve plot rather than isolated single-run impressions.
- A benchmark table comparing baseline and RL-enhanced answer paths.
- Introspection output that helps explain the benchmark results rather than merely reporting them.

23.4 Debugging tips

- If experiments are too noisy, fix the seed and rerun on the same runtime before interpreting the curve.
- If RL seems worse than baseline, inspect whether retrieval is already so narrow that arm differences barely matter.
- If benchmark results improve while citations worsen, you may be over-optimizing for style instead of groundedness.

24 22. Optimization & Scaling

24.1 Intuition

Optimization in Polymind V2 is about preserving quality under constraints, not just reducing runtime. The main cost centers are PDF extraction, embedding batches, multi-arm generation, and repeated experiments. Profiling makes those costs visible, while design tradeoffs explain which knobs are worth turning. For a single-book Colab system, the most valuable optimizations are almost always upstream: better chunking, smarter retrieval, artifact reuse, and disciplined token budgets.

Performance and cost. On a T4, `flan-t5-xl` with 8-bit loading is usually viable, while CPU fallback is noticeably slower but still useful for indexing and light experiments. Embedding stages benefit from batching and often dominate one-time setup cost. Generation dominates repeated question answering cost. Memory usage is most sensitive during model loading and multi-arm answer generation, which is why cache cleanup and task-specific token budgets matter so much. Colab cost is therefore less about raw GPU minutes and more about avoiding unnecessary recomputation.

Design tradeoffs revisited. FAISS beats a managed vector DB here because it minimizes operational surface area and supports single-file persistence. MiniLM beats heavier embedders because retrieval quality is already strong while latency and CPU fallback remain manageable. Bandit RL beats PPO because the action space is tiny and the goal is arm choice, not generative fine-tuning. Chunk size and overlap are tuned for evidence quality, not aesthetics: smaller chunks improve precision, larger chunks improve context, and overlap protects transitions at the cost of duplication. `flan-t5-xl` beats `flan-t5-base` on depth, but only when the runtime can sustain the memory and latency budget.

Scaling beyond one book introduces new pressures. Multi-book corpora need corpus metadata, likely per-book filters, and probably a stronger retrieval stack. Larger evaluation suites need cached experiment results. More complex policies may justify contextual bandits or rerankers. But the foundation does not change: keep expensive work persistent, keep interfaces typed, and keep introspection available at every stage.

24.2 Colab-ready code cell

```
import pandas as pd

if isinstance(answer["profile"], list):
    profile_df = pd.DataFrame(answer["profile"])
```

```
else:
    profile_df = pd.DataFrame()
display(profile_df)

cost_notes = {
    "gpu_runtime": "Best for FLAN-T5-XL generation and repeated experiments",
    "cpu_runtime": (
        "Fine for indexing reloads, artifact inspection, and "
        "lightweight debugging"
    ),
    "one_time_cost": "Chunking + embedding + FAISS build",
    "repeat_cost": "Retrieval + multi-arm generation + reward scoring",
    "best_optimizations": [
        "reuse artifacts",
        "keep top_k realistic",
        "use task-specific token budgets",
        "clean GPU cache between arms",
    ],
}
cost_notes
```

24.3 Expected output

- A profile dataframe showing stage runtimes for a question-answering pass.
- A concise cost model describing where runtime and memory are actually spent.
- A practical optimization checklist rooted in the measured system rather than generic advice.

24.4 Debugging tips

- If question answering is slow even after artifact reuse, inspect whether experiments are repeatedly forcing model reloads.
- If CPU fallback becomes the norm, reduce generation ambitions before weakening the retrieval stack.
- If you are tempted to replace FAISS with infrastructure, make sure the bottleneck is truly indexing or search and not prompt or model latency.

25 23. Future Extensions

25.1 Intuition

Polymind V2 is intentionally ambitious enough to be real while still compact enough to understand. That makes it a strong foundation for future work. The obvious extensions are multi-book corpora, stronger reranking, richer concept graphs, exportable study notes, quiz generation, and better benchmark suites. More advanced policy layers are also possible, especially contextual bandits that condition arm choice on question type or retrieval statistics.

The more important lesson, though, is architectural. Future improvements should not collapse the clarity of the current system. A stronger retriever should still expose inspection hooks. A larger generator should still respect artifact reuse and memory constraints. A better policy should still explain why it preferred one answer style over another. This is the standard that turns experimentation into trustworthy system design.

If you carry this project beyond Colab, keep the same principles. Make state explicit. Persist expensive work. Separate orchestration from logic. Prefer small, inspectable control surfaces over giant hidden notebooks. And when quality drops, debug the data flow before you blame the model. That habit is what actually scales.

25.2 Colab-ready code cell

```
future_extensions = [  
    "Add multi-book indexing with per-book filters and corpus metadata",  
    "Introduce a reranker over retrieved chunks before prompting",  
    "Upgrade the bandit to a contextual bandit using retrieval diagnostics as context",  
    "Export notes, flashcards, and chapter quizzes from grounded evidence",  
    "Add richer evaluation suites and cached experiment tracking",  
]  
future_extensions
```

25.3 Expected output

- A clean list of next-step extensions grounded in the current architecture.
- A sense of how the present system can grow without being replaced wholesale.

- A closing reminder that the design discipline is as valuable as the current feature set.

25.4 Debugging tips

- If an extension requires bypassing the current inspection APIs, it probably needs a cleaner interface first.
- If a new feature cannot persist its own expensive artifacts, it will likely make the Colab experience worse.
- If a future change obscures the evidence trail from retrieval to answer, stop and redesign before adding more complexity.

```
\cleardoublepage \appendix \part*{Appendices} \addcontentsline{toc}{part}{Appendices}
\markboth{Appendices}{Appendices} \cleardoublepage
```

26 Appendix A. Mastery Casebook And Diagnostic Walkthrough

26.1 A.1 Why Mastery.pdf Is A Good Stress Test

Mastery.pdf is not just a convenient example book. It is useful because it contains exactly the kinds of structural properties that reveal whether a deep reading pipeline is genuinely robust. The opening pages contain front matter and layout conventions that can fool naive chapter detectors. The body text mixes storytelling, analysis, and advice, which means retrieval must handle both episodic and thematic queries. The book also revisits central themes such as apprenticeship, observation, social intelligence, power, experimentation, and creative independence across multiple regions. That repetition is good for a reader, but it creates a harder retrieval task because semantically similar language can appear in several chapters with different emphasis.

This is why the guide keeps coming back to Mastery when it explains diagnostics. A book with cleaner chapter markers and shorter, more isolated themes would let the system look stronger than it really is. Mastery forces the implementation to earn its claims. If heading extraction is sloppy, front matter leaks into the chapter map. If chunking is unstable, overlapping themes produce repeated evidence windows. If retrieval lacks diversity, it may collapse into one chapter that happens to mention a popular keyword. If citations are not verified, the generator can invent confident-looking references that do not correspond to the retrieved set. Each of those problems is visible in this book, which makes it a good benchmark and a good teaching source at the same time.

Another reason the book is useful is that the questions people naturally ask about it vary in shape. Some are local and factual: What is the apprenticeship model? Some are interpretive: Why does Greene care so much about social intelligence? Some are synthetic: How does patience connect to experimentation? Some are comparative: What changes between early apprenticeship and the creative-active phase? Because the question distribution is varied, the answer arms are forced to compete on meaningful terrain rather than on one repetitive style of prompt. A concise arm may win narrow factual questions. A teacher-style arm may do better on chapter-level explanations. A synthesizer may shine when themes must be connected across distant evidence.

The casebook sections below describe how those traits surface during system development. They are not abstract warnings. Each one reflects a failure mode that appears quickly if you weaken one of the updated design choices in Polymind V2. Treat them as concrete debugging narratives. When the system later behaves strangely on a different book, these cases provide a vocabulary for what you are seeing and where to look first.

26.2 A.2 Front-Matter Chapter False Positives

The first recurring failure case in Mastery comes from the opening pages. The front matter includes title pages, section labels, and typographic patterns that can look like chapter headings when you only use uppercase regexes over the raw text. A naive detector often finds headings that look plausible in isolation but are not actual chapter boundaries in the reading flow. When that happens, the system does not fail immediately. It continues ingesting text, but the chapter IDs begin to drift away from the semantic structure of the book. That produces misleading chunk metadata, and the downstream symptoms show up in places that look unrelated: chapter-level summaries feel unbalanced, diversity constraints skip the wrong chunks, and citations cite chapter numbers that are internally consistent but narratively wrong.

The updated detector solves this in three layers. First, it still uses regexes, because regexes are cheap and effective for conventional headings. Second, it adds font-derived candidates from `page.get_text("dict")`, which is often the best signal on professionally typeset PDFs. Third, it deduplicates candidates in a character window rather than assuming exact span identity. That matters because the same visible heading can be discovered through slightly different offsets depending on whether the signal came from text extraction or font spans. The windowed deduplication rule treats those near-matches as one semantic event.

When you diagnose this issue in practice, begin with three questions. Do the earliest detected chapter titles look like real chapters or like cover-page artifacts? Do the first few sections have meaningful lengths or tiny fragments? Do chapter IDs later in the book appear shifted relative to your expectations? If the answer to any of those is yes, the fix should happen in heading detection, not in chunking or retrieval. A common mistake is to notice poor retrieval later and to compensate with larger `top_k` or broader prompts. That only buries the structural issue under more evidence. The right move is to inspect the early chapter candidates and correct the structure at the source.

There is also a process lesson here. Engineers often underestimate metadata correctness because the visible output still looks fluent. But in a reading system, wrong metadata is a silent model of the book. Every later component inherits it. That is why chapter inspection belongs near the start of the workflow and why the guide treats it as a first-class diagnostic stage rather than a preprocessing footnote.

26.3 A.3 Abbreviation Splitting Around Dr. And U.S.

The second casebook issue comes from sentence boundaries. Mastery includes names, initials, abbreviations, and punctuation patterns that break simplistic regex tokenizers. If the splitter sees Dr. or U.S. and assumes a sentence ended, it creates fragments that are shorter than they should be and shifts the semantic meaning of the next chunk boundary. The immediate symptom is often a chunk

preview that looks slightly odd: a chunk beginning with a lowercase word, or a sentence that seems to have been broken after an honorific. But the bigger consequence is retrieval distortion, because dense embeddings lose some of the coherence that comes from properly preserving full sentences.

Polymind V2 treats NLTK sentence tokenization as the default and keeps a robust fallback for environments where the tokenizer is unavailable. The fallback protects a set of high-value abbreviations, rewrites multi-capital patterns such as U.S. before splitting, and then restores punctuation after segmentation. This is not as strong as a dedicated language model tokenizer, but it is far better than relying on punctuation-plus-capital heuristics alone. More importantly, it is predictable and testable. The unit tests explicitly check the Dr. and U.S. cases because they stand in for a broader class of sentence edge failures.

The casebook lesson is that sentence splitting should be evaluated by reading the resulting chunks, not only by running the function successfully. A tokenizer can return a list of strings and still be wrong in the ways that matter most for a retrieval pipeline. The guide therefore recommends a diagnostic loop that is small but revealing: inspect the first few chunks from a known problematic region, inspect the last chunk of a chapter, and inspect a chunk around one of the book's abbreviation-heavy passages. If those look clean, the splitter is probably stable enough for the rest of the pipeline.

When the splitter is not stable, resist the temptation to compensate with much larger chunks. Larger chunks sometimes hide the visible symptom because the fragment is surrounded by enough other sentences to look coherent. But the underlying offset and overlap logic still suffers. It is better to fix sentence boundaries and keep chunk size aligned with the intended retrieval budget.

26.4 A.4 Same-Chapter Retrieval Collapse Without Diversity

Retrieval collapse is one of the most educational failures in the whole system. Imagine a query about leadership and social intelligence. Mastery has several passages related to those concepts, but a plain dense retriever may return five very similar chunks from the same chapter because those passages happen to be the closest in embedding space. The result looks respectable at first glance. The chunks are relevant, the scores are high, and the prompt is grounded. But the evidence set is narrower than the question deserves, so the answer can end up repeating one local argument while ignoring complementary material elsewhere in the book.

The upgraded retriever counters that in two ways. Cosine deduplication removes overlapping chunks that would otherwise consume the prompt budget with repeated language. Chapter diversity enforcement then limits how many final chunks can come from the same chapter. The combination matters. Deduplication alone can still leave five distinct but same-chapter chunks. Diversity alone can still allow near-duplicates from adjacent windows. Together they create an evidence set that is both compact and distributed.

`inspect_retrieval()` is the tool that makes this visible. A good inspection run shows raw top-k hits, selected hits, dedup events, and diversity events. That lets you answer questions like: was a useful chunk lost because it was too similar to an already chosen chunk, or because the chapter limit was reached? Those are different interventions. If good evidence is being lost to the deduplication threshold, adjust overlap or thresholding. If good evidence is being lost because a whole chapter dominates the raw results, HyDE expansion or better query rewriting may be the right next move.

The deeper lesson is that relevance alone is not the goal of retrieval in long documents. Coverage matters too. A reading system must often recover the shape of an argument, not just the single closest passage. Diversity is therefore not a luxury feature. In book QA, it is often a prerequisite for answers that feel like reading intelligence instead of passage parroting.

26.5 A.5 Citation Hallucination Before Verification

Once the generator begins citing chunks, it becomes tempting to treat citation formatting as proof of grounding. Mastery is a good reminder that this is not enough. Even a careful prompt can produce an answer that cites a chunk ID that looks right but was never in the retrieved set. The generator is not malicious; it is simply pattern-completing. If the prompt repeatedly shows citations like [Chapter 03 | Chunk 014], the model may synthesize that style even when it is no longer tethered to the actual evidence window.

Polymind V2 treats citation verification as a post-generation safety step, not as an optional polish pass. The system parses citation patterns, checks them against the retrieved results, and strips any invalid references before the final answer is returned. This has two benefits. It prevents visually convincing but unsupported references from reaching the user, and it turns citation validity into a measurable reward component. That means the RL layer can prefer answer arms that stay grounded without rewarding fabricated confidence.

The diagnostic trick with this failure case is to compare three things together: the retrieved chunk set, the raw answer text before verification, and the final verified citations. If the raw answer consistently invents nearby chunk IDs, the prompt may need stronger citation instructions or the answer arm may be drifting toward synthesis without enough grounding pressure. If the raw answer rarely cites at all, the prompt may be too vague or the answer budget too short. The goal is not to maximize citation count blindly. It is to maximize trustworthy evidence references under the actual retrieval budget.

This case also illustrates a general system-design principle: a model should not be trusted to validate its own control tokens. The same lesson appears in tool use, JSON output, and function calling. When the system depends on a small set of special markers, external validation is cheap insurance.

26.6 A.6 Bandit Preference Drift When Reward Normalization Is Wrong

The final Mastery casebook issue ties together answer style, reward design, and policy behavior. If reward normalization is weak, the bandit can begin preferring an answer arm for the wrong reason. A longer teacher-style answer may accumulate more keyword hits simply because it uses more words. A more speculative synthesizer answer may be punished by a lexical hallucination metric even when it is semantically faithful. Over time, those biases create policy drift: the bandit appears to be learning, but it is really amplifying defects in the reward function.

The updated reward design fixes this in three ways. Grounding is measured through embedding similarity, not exact lexical overlap. Keyword overlap is normalized by answer length, so verbosity does not dominate. Divergence is measured against a held-out non-selected chunk rather than against the absence of surface-word reuse. The result is a reward that still remains lightweight enough for notebook use but is much less biased against paraphrase and synthesis.

The practical debugging pattern is to inspect a small batch of questions and compare reward tables across arms. If one arm always wins by a small margin and the reasons look suspiciously similar across every question, the reward function is probably too rigid. If the preferred arm and selected arm diverge constantly, the policy may be over-exploring or the reward surface may be noisy. If epsilon never decays into a more exploitative regime, learning cannot stabilize. These are not abstract RL concerns. They directly affect the quality of the answer the reader sees.

For Mastery, this issue becomes visible most quickly on mixed conceptual questions such as leadership plus social intelligence or patience plus experimentation. Those questions reward answers that connect ideas without losing book grounding. A broken reward function often punishes exactly that kind of answer. A healthier reward design allows synthesis to win when it is truly the best grounded explanation, not merely when it is the most verbose candidate.

27 Appendix B. Benchmark And Experiment Cookbook

27.1 B.1 Building A Useful Mastery Benchmark Set

A benchmark fixture for a book-sized system should not behave like a trivia quiz. If every question can be answered from a single short passage, the benchmark only measures the easiest case. The `mastery_eval_set.json` fixture in this project is designed around a broader philosophy. It includes single-hop questions, but it also includes cross-chapter synthesis prompts, follow-up style prompts, and concept-linking prompts that reveal whether the system can maintain thematic coverage without drifting away from the book. That variety is what makes the benchmark useful for both retrieval engineering and answer selection.

Each fixture entry contains a question, a small list of expected chapter IDs, a tag set, and expected keywords. That structure is intentionally lightweight. The goal is not to create a brittle gold standard that breaks every time the chapter detector changes by one offset. Instead, the fixture gives the notebook enough structure to measure chapter recall, qualitative topical relevance, and answer coverage while remaining resilient to small implementation details. In a Colab setting, that tradeoff is important because benchmark maintenance should remain cheap enough that readers actually use it.

Another design choice is to include questions that are close in surface form but different in reasoning demand. For example, asking what apprenticeship is and asking how apprenticeship relates to experimentation are nearby prompts in language, but they require different evidence distributions. Those pairs are valuable because they stress the retrieval layer. If the same chapter keeps winning both queries, something about the retrieval stack may be too narrow. If the same answer arm keeps winning both, the reward layer may not be sensitive enough to question type.

The fixture should also help readers reason about failure, not just success. That is why tags matter. If a retrieval change improves single-hop questions but hurts cross-chapter prompts, the benchmark should make that visible. If RL seems to improve average reward but follow-up questions get worse, the benchmark should make that visible too. A good fixture partitions the workload into meaningful subsets rather than collapsing everything into one average.

27.2 B.2 Running A Top-k Sweep

The top_k sweep is the fastest way to make retrieval tradeoffs concrete. In the simplest version, you choose a fixed question slice from the benchmark set, run `system.ask()` with several top_k values, and record a small metric bundle: maximum candidate reward, number of verified citations, selected arm, and answer length. The point is not that top_k alone determines quality. The point is that too little evidence and too much evidence fail in different ways. Low top_k often improves precision but hurts coverage. High top_k may broaden context but inflate prompt noise unless deduplication and diversity stay strong.

When analyzing the sweep, avoid relying on only one metric. A higher average reward may coincide with a drop in citation validity or chapter diversity. Longer answers may look better subjectively while actually becoming more repetitive. This is why the notebook plot is only one view. The table of per-question results matters just as much. Read a few answers at the low and high ends of the sweep. Ask whether the answer became more complete, or merely more wordy. Ask whether the cited chunks still span the right parts of the book. Those qualitative checks keep the experiment honest.

There is also an operational angle. Increasing top_k changes prompt length, which changes generation latency and sometimes memory pressure. In a Colab-first system, that matters. A top_k that is slightly better in average reward but meaningfully worse in latency may not be the right default for an interactive reading assistant. The cookbook mindset is therefore to measure quality and cost together, not as separate tracks.

27.3 B.3 Running A Chunk-Size Sweep

Chunk-size experiments are slower than top_k sweeps because they require rebuilding the chunk pool and embeddings, but they are often more informative. Chunk size affects almost every later stage. Smaller chunks increase retrieval precision and make citations feel sharper, but they can fragment ideas that need several sentences to make sense. Larger chunks preserve local discourse better, but they blur boundaries and increase overlap-induced redundancy. Because both effects are real, a chunk-size sweep is one of the best ways to teach retrieval engineering through a concrete document.

The notebook recipe in this project clones `PolymindConfig` with several target word counts, rebuilds the system, and then evaluates the same benchmark question under each setting. The minimal metrics are chunk count, top reward, and citation count, but the real value comes from inspecting the resulting chunks themselves. When a larger chunk size wins, check whether it did so because the answer needed more intact context or because the smaller setting was suffering from bad sentence boundaries. When a smaller chunk size wins, ask whether the gain came from sharper retrieval or from accidental chapter over-segmentation.

Chunk-size sweeps also reveal why code structure matters. Because the system persists artifacts, you can afford to run several rebuilds and keep their profiles. Without persistence, a chunk-size study in Colab would be annoying enough that most readers would skip it. That is exactly the kind of user behavior that system design should anticipate. Good tooling is not only about faster code; it is about making the right experiments cheap enough to actually happen.

27.4 B.4 Measuring Temperature Versus Reward

Temperature is one of the most overused and least contextualized knobs in casual LLM work. In Polymind V2 it has a more disciplined role. Temperatures are tied to answer arms, not treated as one global magic slider. The point is to give the system distinct candidate styles that can then be compared under the same retrieved evidence. This is why the temperature-versus-reward plot in the notebook matters. It is not searching for the universally best temperature. It is checking whether the intended answer styles behave as expected on the benchmark questions.

When you run this study, resist the urge to interpret the plot too literally. The synthesizer arm may have a higher temperature, but its behavior depends on more than sampling noise. Prompt instruction, reward normalization, and the breadth of the retrieved evidence all shape whether it succeeds. The plot therefore works best when read alongside the actual reward breakdowns and a few sample answers. If the synthesizer loses every time, the problem might be temperature, but it might just as easily be overly narrow retrieval or a reward function that still slightly favors shorter lexical overlap.

The notebook implementation uses the existing `ask()` payload and joins arm names back to `config.answer_arms`. That design is intentional. It keeps the benchmark logic notebook-owned while relying on stable system outputs rather than hidden internal state. This is the pattern to follow for later experiments as well: expose structured metrics from the system, but keep experiment orchestration in the notebook where the reader can see and change it.

27.5 B.5 Comparing `flan-t5-base` And `flan-t5-xl`

Model comparison should answer an operational question, not a prestige question. For this handbook the relevant question is simple: what quality gain does `flan-t5-xl` buy on a T4 or CPU fallback path, and when is that gain worth the latency and memory cost? The benchmark cell handles this by instantiating systems with different generator defaults while leaving the retrieval stack unchanged. That isolates the effect of generator capacity as much as possible in a notebook setting.

A useful model comparison records at least four things: reward, verified citation count, answer length, and latency. Reward tells you whether the larger model is earning its extra capacity under the

system's groundedness criteria. Citation count checks whether the larger model stays disciplined or drifts toward cleaner language with weaker source anchoring. Answer length gives a crude proxy for how the model uses its budget. Latency matters because a notebook reading assistant must still feel interactive enough for repeated questioning.

The comparison is especially instructive on deep or synthetic questions. Base models often remain adequate on narrow factual prompts because retrieval is doing most of the hard work. The difference becomes clearer when the model must connect several ideas without breaking grounding. This is why the benchmark fixture includes both narrow and broad prompts. A model comparison built only on short questions would understate the value of the larger generator.

27.6 B.6 Retrieval Ablations

Retrieval ablations are where the value of the upgraded stack becomes easiest to defend. The notebook compares a baseline dense retrieval mode against increasingly richer configurations: +HyDE, +HyDE + dedup, and the full stack with chapter diversity. The goal is not to prove that every extra mechanism helps every question. The goal is to expose which kinds of improvements come from which part of the retrieval design. HyDE often helps vague conceptual questions. Dedup helps overlapping chunk pools. Diversity helps thematic prompts that otherwise collapse into a single local region.

Read the ablation table as a story about evidence composition. If the full stack selects fewer chunks but yields better answers, that is often a sign that dedup and diversity cleaned the prompt budget. If HyDE increases raw retrieval quality but also increases same-chapter collapse, that tells you the expansion is working but the evidence still needs balancing. The ablation study therefore becomes a practical lesson in layered retrieval design rather than a single winner-take-all comparison.

One subtlety is worth noting. Because the notebook uses cloned configs to run ablations, the experiment can stay close to the public system surface. That is a good habit. A benchmark that depends on mutating internal objects or bypassing public APIs becomes harder to maintain and easier to misread. In other words, the experiment design should respect the same interface boundaries as the production workflow.

27.7 B.7 Cached-Versus-Cold Profiling

Cold runs and cached runs answer different questions. A cold run tells you how expensive it is to bring a new book into the system. A cached run tells you how usable the system will feel after a normal Colab disconnect or notebook restart. Both matter. Many educational systems look great in

the first category and poor in the second because they quietly require substantial recomputation after every session reset. Polymind V2 is explicitly designed to do better than that.

The profiling recipe is simple: time a forced recompute, then time a `load_from_artifacts()` path against the same project root. The numbers will vary by runtime, but the relationship should be dramatic. If the cached path is only a little faster than the cold path, either the artifacts are incomplete or the resume logic is not respecting them. If the cached path is fast but later answers look different, the metadata may be out of sync with the current config. The right response is not to ignore the difference, but to treat it as a contract failure and inspect the saved state.

This experiment is also a reminder that production quality in notebooks is not only about model speed. Operational smoothness matters. A user will often judge the reliability of a system by whether it picks up where it left off without surprises. In that sense, cached-versus-cold profiling is a user-experience benchmark as much as it is a performance benchmark.

28 Appendix C. Failure Playbooks And Recovery Paths

28.1 C.1 Ingestion And Chaptering Failures

When ingestion fails, the visible symptom is often later than the actual cause. The book may load. The system may build sections. Chunks may be generated. Only when retrieval feels off or chapter labels look strange does the operator notice that something is wrong. The right mindset is to treat ingestion failures as structural faults. They do not usually crash the notebook. They distort the shape of the book in ways that every later component inherits.

The first playbook step is to inspect the cleaned text and the detected chapters side by side. If the first few chapter titles look like cover matter, acknowledgments, or part labels rather than real content transitions, the detector is too permissive. If the chapter offsets are bunched too closely together, heading deduplication is probably too weak. If the chapter list is implausibly short, the regex patterns or font threshold may be too strict. Each symptom suggests a different intervention. This is why the chapter detector remains simple and explainable. A fully opaque heading classifier might sometimes perform better, but it would be much harder to debug when the book structure looks wrong.

Recovery is straightforward when the failure is caught early. Re-run ingestion, inspect the heading candidates, and adjust only the chaptering layer. Do not touch chunk size, retrieval parameters, or prompt design in response to a chaptering fault. That is a classic example of treating downstream symptoms instead of upstream causes. In a well-factored system, the cheapest recovery is usually the earliest recovery.

28.2 C.2 Chunking And Offset Failures

Chunking failures come in several recognizable forms. Some are visible in the inspection table: short chunks everywhere, overlapping offsets, low sentence counts, or repeated previews. Others only become obvious when citations or retrieval behave oddly. Because of that, the playbook begins with chunk inspection even when the user complaint sounds like a generation issue.

If many chunks are shorter than expected, ask whether the cause is genuine book structure or tokenization failure. A chapter ending with one short remainder is normal. Whole runs of short chunks usually are not. If offsets move backward or overlap too aggressively, treat that as a source-tracing failure and stop using citations until it is fixed. If previews show repeated language across

adjacent chunks, compare overlap size with retrieval dedup settings. Sometimes the system is doing exactly what it was asked to do and the problem is simply that the chosen overlap is too generous for the benchmark questions you care about.

Recovery in this layer usually means changing one of three things: sentence segmentation, target chunk size, or overlap. Avoid changing more than one of those at once. The chunk-size sweep in the notebook exists partly to enforce that discipline. It is much easier to understand the effect of a chunking change when you isolate it and keep the rest of the system stable.

28.3 C.3 Embeddings, FAISS, And Resume Failures

The embedding and index layers introduce a different family of failures: runtime limits, device mismatch, and stale artifacts. On Colab, the most common visible symptom is an out-of-memory failure during embedding or generator loading. On reload, the most common symptom is an index that exists on disk but does not behave the same way under the current runtime as it did under the previous one. These are the problems that convince many people that notebooks are inherently fragile, when the real issue is that too much state was left implicit.

The updated system handles these failures by design. Embedding is explicit and batched. CUDA OOM triggers a CPU fallback path. The FAISS index is normalized on load. Artifact presence is checked before reuse. The recovery playbook therefore begins with a simple audit: which artifact files exist, what config generated them, and what device is active now? If those answers are consistent, reload should work. If they are inconsistent, rebuild the affected layer instead of trying to patch around it.

There is a broader lesson here about persistence. Saving files is not enough. You also need to know what those files mean. That is why `metadata.json` matters. Without configuration context and stage profiles, persisted files are only half an artifact contract. With that context, resume logic becomes debuggable instead of mysterious.

28.4 C.4 Retrieval Failures

Retrieval failures often masquerade as model failures because the answer is the part the user sees. The playbook reverses that habit. When an answer looks weak, inspect retrieval first. Look at the rewritten query, the HyDE expansion, the raw top-k hits, the selected results, dedup events, and chapter counts. That tells you whether the issue is query formulation, semantic search, evidence redundancy, or coverage.

A narrow evidence set usually points toward one of three causes: the question was not rewritten clearly enough, the query expansion is disabled or weak, or the diversity setting is too lax. A

noisy evidence set usually points toward the opposite problem: too much retrieval breadth, weak deduplication, or chunks that are too large and semantically mixed. The right fix depends on which pattern appears in the inspection output. This is why retrieval debugging deserves a public API rather than a print statement buried inside a helper function.

Recovery should move from the least expensive change to the most expensive. Rewrite and expansion fixes are cheap. Dedup and diversity tuning are cheap. Chunk-size changes are more expensive because they require rebuilding artifacts. Book-level cleaning or chaptering changes are most expensive because they change the substrate of everything else. A good operator moves through that ladder in order instead of jumping straight to the deepest rebuild.

28.5 C.5 Generation And Citation Failures

Generation failures divide into three broad categories: unsupported drift, under-explanation, and citation unreliability. Unsupported drift is when the answer sounds plausible but wanders beyond the retrieved evidence. Under-explanation is when the answer remains technically grounded but fails to teach the concept with enough depth for the question. Citation unreliability is when the answer style looks grounded because it includes citations, but the citations are thin, absent, or fabricated.

The recovery playbook starts by reading the retrieved evidence next to the final answer. If the answer is too narrow, the problem may actually be retrieval. Assuming retrieval is healthy, the next diagnostic is the reward table. If the teacher arm consistently loses even when its answers are stronger to a human reader, the reward function may still slightly favor brevity or lexical overlap. If the synthesizer drifts, check whether the evidence set is broad enough to justify synthesis and whether the divergence penalty is catching held-out-chunk affinity.

Citation recovery is more direct. Compare raw and verified citations. If many raw citations are stripped, prompt discipline or arm behavior is weak. If few or no citations are present even on evidential questions, the prompt may be too vague or the answer budget too short. As always, fix the narrowest plausible cause first.

28.6 C.6 RL And Memory Failures

RL and memory failures are less obvious because they unfold over time. The first few questions may look fine. The drift appears after repeated interactions, when the bandit develops habits and the memory summary begins influencing retrieval. This is why the playbook focuses on trend inspection rather than single examples.

For RL, the key questions are: Is epsilon decaying? Are pulls being recorded for all arms? Do the reward components look meaningfully different across questions? Does the preferred arm diverge from the selected arm more often than expected? Those questions tell you whether the policy is learning, wandering, or simply reflecting a broken reward surface. For memory, the key questions are: Is the summary factual and compact? Are rewritten follow-ups becoming self-contained or bloated? Are later retrievals clearly influenced by old turns in ways that still match the user's intent?

Recovery here requires restraint. If memory is making retrieval noisy, shorten the summary and rewrite prompt before changing retrieval itself. If RL looks unhelpful, inspect reward normalization before reducing exploration. Time-based drift invites over-correction because the failure feels diffuse. The right response is to keep the diagnostic surface structured and to change one control surface at a time.

29 Appendix D. Artifact, API, And Notebook Reference

29.1 D.1 Artifact Contracts

Polymind V2 persists a small set of artifacts because each one represents a stable boundary in the data flow. `chunks.json` is the human-readable record of what the system believes the book has become after chunking. It is also the easiest place to inspect whether chapter IDs, offsets, and previews look sane. `embeddings.npy` is the vectorized form of that same substrate. It is not useful to read directly, but it is critical for reuse and for ensuring that FAISS is searching over the same chunk pool the notebook thinks it is. `metadata.json` binds those files to the config and profiling context that created them.

The index artifact is distinct from the embeddings artifact because the search structure has its own operational concerns. That is why `faiss.index` is saved and loaded through dedicated helpers rather than treated as just another binary blob. Finally, `rl_bandit_stats.json` tracks the evolving preference surface over answer arms. That file matters even when the book artifacts are unchanged, because a question-answering session is not only a retrieval workload. It is also a small learning process over the user's workload.

Treating artifacts this way creates a powerful debugging pattern. When something looks wrong, ask which contract is likely broken. If the answer involves chapter labels or citations, inspect `chunks.json`. If the answer involves reload speed or FAISS behavior, inspect the index and metadata. If the answer involves arm choice drift, inspect the policy stats file. This is much faster than treating the whole system as one opaque state.

29.2 D.2 Dataclass Reference

The project uses dataclasses because they make the pipeline legible. `BookSection` represents the book after structure has been detected but before chunking. It holds the chapter title, the section text, and source offsets. `ChunkRecord` represents the retrieval unit and adds sentence counts, word counts, and chunk metadata. `RetrievalResult` represents the evidence as it comes back from search, including the score, rank, and source reason. Those three classes together tell the story of how the document became a searchable substrate.

On the answer-selection side, `AnswerCandidate` stores one arm's prompt, answer, temperature, token budget, citations, and metadata. `RewardBreakdown` stores the components used to choose among

those candidates. `BanditArmStats` stores the persistent learning state of the RL layer. `MemoryTurn` stores the conversational trace that later gets compressed. The inspection dataclasses then expose those states back to the notebook in a structured form. This is what keeps the notebook from devolving into ad hoc dictionaries and print statements.

One subtle benefit of this design is that the dataclasses create a shared mental model between prose and code. The handbook can refer to a retrieval result or a reward breakdown as a real typed entity rather than an informal idea. That makes the documentation more precise and the experiments easier to interpret.

29.3 D.3 PolymindSystem Call Patterns

There are only a handful of workflows that most readers need. The first is the index-or-load pattern. Start with `index_book()` when the artifacts are absent or when you intentionally want a fresh build. Use `load_from_artifacts()` or the `load_book()` alias when artifacts already exist and you want to resume quickly. The second pattern is the question-answering loop. Use `ask()` for normal questions, then inspect rewards and memory if you are tuning quality. The third pattern is the study loop. Use `summarize()` for level-specific summaries and `link_concepts()` for cross-theme synthesis.

The introspection methods fit around those workflows rather than replacing them. Use `inspect_chunks()` right after indexing. Use `inspect_retrieval()` whenever a question seems under-supported or strangely focused. Use `inspect_rewards()` when two answer styles feel close and you want to understand the tie-break. Use `inspect_memory()` when follow-up rewriting begins to shape retrieval in ways you need to audit. Those methods are cheap enough to use frequently, which is why they matter. An inspection API that feels too expensive or awkward will not be used consistently.

This call pattern is also what the notebook mirrors. The notebook is not supposed to teach a second interface. It is supposed to model good use of the first one.

29.4 D.4 Notebook And Guide Synchronization

The notebook and guide should be read as complementary views over the same system, not as separate products. The guide provides the reasoning, design tradeoffs, case studies, and recovery paths. The notebook provides the runnable sequence, benchmark cells, and quick visual checks. The package provides the actual logic. Drift between the three is one of the easiest ways for a technical guide to become frustrating. That is why the repo includes a notebook builder, guide exporter, and validation script.

The synchronization rule is simple. If a code pattern matters enough to teach, it should live in the package or a build helper. If a code pattern matters enough to demonstrate, it should appear in the notebook with safe line wrapping. If a code pattern matters enough to explain, it should appear in the guide with a prose context that matches the current implementation. That three-way alignment is the closest thing a handbook has to a production release process.

29.5 D.5 Guide Maintenance Checklist

When the system changes, update the artifacts in a deliberate order. First, make the package change and verify it with tests. Second, update the notebook builder so the runtime walkthrough reflects the new behavior. Third, update the guide source so the narrative and code block match the implementation. Fourth, rebuild the notebook and export the PDF. Fifth, run the validation script. This order ensures that prose never gets ahead of code and that the PDF remains a trustworthy representation of the current repo state.

The maintenance checklist may sound procedural, but it has a strategic purpose. Technical handbooks age fastest when narrative and execution diverge. By treating the guide as part of the build output rather than as a one-time document, the project stays honest over time.

30 Appendix E. Colab Operations, Cost, And Production Readiness

30.1 E.1 Runtime Profiles And What They Mean

A Colab runtime is not just a machine. It is a contract with uncertainty. The GPU may be present or absent. Available memory may vary. Session lifetime is limited. Local disk is ephemeral. Any guide that ignores those facts produces code that is technically correct but operationally fragile. This appendix treats those runtime properties as design inputs rather than as afterthoughts.

The first useful distinction is between one-time costs and repeated costs. One-time costs include extraction, chunking, embeddings, and FAISS index building for a new book. Repeated costs include question answering, summaries, concept linking, and experiments over an existing artifact set. The persistence strategy is what separates those categories. When artifacts are reused correctly, a reader can spend most of their Colab time on the repeated costs, which are the ones that actually teach retrieval engineering and answer-selection behavior.

The second distinction is between GPU-sensitive and CPU-tolerant work. Generator loading and multi-arm answer generation are the most GPU-sensitive parts of the system. Ingestion, cleaning, chunking, artifact inspection, and many structural tests are CPU-tolerant. Embeddings sit in the middle. They benefit from GPU acceleration, but the batching and fallback logic allow them to complete on CPU when necessary. This profile matters because it shapes how you react to a runtime change. Losing the GPU does not mean the whole project is blocked. It means the workflow should shift toward indexing, inspection, and lighter experiments until the heavier generation path is available again.

30.2 E.2 Memory Hygiene On A T4

The T4 is generous enough for serious experimentation and small enough to enforce discipline. `flan-t5-xl` only becomes comfortable when quantized and when the pipeline avoids unnecessary duplication. This is why the guide keeps repeating three ideas: load the generator safely, keep token budgets tied to task type, and clear the CUDA cache between answer arms. None of those steps is glamorous, but they make the difference between a reliable session and a notebook that fails midway through a benchmark loop.

One practical rule is to treat multi-arm generation as the dominant memory event. Retrieval may handle several chunks, but those chunks are small compared to a large generation model producing multiple outputs in sequence. If a session is unstable, reduce the depth of the most expensive operations first. Lower token budgets for deep tasks, reduce benchmark batch size, or temporarily compare two arms instead of three. Do not immediately reach for a smaller model unless the session remains unstable after those cheaper interventions.

Another rule is to avoid redundant model loading. The notebook should create the system object once and reuse it. Re-instantiating the generator in several cells is a quiet way to waste memory and confuse latency measurements. This is one more reason the public PolymindSystem surface matters: it gives the notebook a place to concentrate state rather than rebuilding it ad hoc across cells.

30.3 E.3 Cost Awareness Without Paralysis

Colab cost in this project is partly monetary and partly cognitive. Monetary cost shows up when longer sessions, higher-end runtimes, or repeated rebuilds consume more compute time than expected. Cognitive cost shows up when the environment is annoying enough that readers stop running the experiments that would actually teach them something. The design aim is to reduce both.

Artifact reuse is the biggest cost lever because it protects both kinds of cost at once. Once the book is indexed, experiments become lightweight enough to feel interactive. The next biggest lever is experiment discipline. Use a benchmark slice when exploring an idea. Save the full benchmark set for stronger comparisons. Profile cold and cached runs once, not every session. Compare models on representative prompts instead of every possible question. Those habits keep the notebook usable without diluting the engineering lessons.

Cost awareness should not become fear. A common anti-pattern is to avoid running the most revealing experiments because they look expensive. The better pattern is to build the system so those experiments are targeted, persistent, and comparable. That is what this repo is trying to do.

30.4 E.4 Session Reset Playbook

A Colab reset should feel inconvenient, not catastrophic. The recovery playbook is simple. Mount Drive. Recreate the config. Rebuild the system object. Call `load_from_artifacts()`. Verify that the benchmark fixture is present. Ask one known question and inspect the retrieval output. If those steps succeed, the session is effectively back to where it was before the reset, minus any in-memory-only conversation state you intentionally did not persist.

If the load path fails, inspect the artifact directory before rebuilding. A missing index file suggests an interrupted or incomplete previous run. A present index with absent metadata suggests a contract violation in the save path. A present metadata file with radically different config values suggests that the current session is pointed at the wrong project root or has incompatible expectations. Rebuilding immediately without that quick audit can destroy useful debugging evidence.

This playbook also highlights an intentional design boundary. Conversation memory is session-local in the current build, while retrieval artifacts and bandit stats are persisted. That balance keeps resume logic simpler while still preserving the most expensive and important artifacts. If you later decide to persist memory turns as well, do so deliberately and extend the artifact contract rather than letting notebook state leak into persistence by accident.

30.5 E.5 Production Readiness Checklist

It is easy for a project to feel production-like because it has many components. Real readiness is more specific. The following questions are a better bar. Can the system be restarted without recomputing the book? Can a human inspect chunk quality, retrieval evidence, reward choice, and memory summary without reading the whole codebase? Can the notebook benchmark the main retrieval and generation tradeoffs without editing internal modules? Can the guide and notebook be rebuilt from source without hand-curated PDF edits? Can the final artifact set tell you what happened when something goes wrong?

If the answer to those questions is yes, then the system is not merely a demo. It is an inspectable, reproducible prototype. That is a strong place to stop for a single-book Colab system. Production does not have to mean cloud infrastructure and dashboards on day one. It can mean that the current operating loop is stable, diagnosable, and extendable.

The checklist is especially important when you are tempted to add features. More models, more retrieval tricks, and more output modes all sound attractive. But a feature that makes artifact reuse harder, inspection murkier, or evaluation less trustworthy is usually a step backward. Readiness is not only about what the system can do. It is about what the system can explain about itself.

30.6 E.6 Scaling Beyond One Book

The path from one book to many books is not just a matter of indexing more PDFs. A multi-book system introduces corpus metadata, filtering decisions, and more complex retrieval tradeoffs. Without explicit book identifiers, chapter namespaces, and benchmark partitioning, the system may retrieve semantically related passages from the wrong book and still look superficially relevant. That is why this handbook treats single-book quality as a prerequisite rather than a toy problem.

The same principle applies to policy learning. A bandit that works well on one book and one benchmark distribution may not generalize cleanly to a mixed corpus. At that stage, contextual bandits or reranking layers might make sense. But the reason to wait is not that the ideas are advanced. It is that you should first understand what the current introspection surface is already telling you about the failure modes of the simpler system. Good scaling decisions are usually based on known bottlenecks, not on abstract completeness.

If you do scale, keep the core habits from this project: explicit artifacts, artifact-aware validation, public inspection APIs, and experiment fixtures that partition the workload by behavior rather than treating every prompt as the same kind of task.

30.7 E.7 Final Review Checklist Before Shipping

Before you call the handbook complete, perform a final review that mirrors the user experience of someone building from scratch. Confirm that the guide reads coherently in PDF form and that no code block clips the page. Confirm that the notebook rebuilds from its script and that the line lengths remain safe for PDF export. Confirm that the benchmark fixture exists and covers multiple question types. Confirm that tests pass. Confirm that the validation script succeeds. Confirm that the rendered PDF is comfortably above ninety pages and still feels purposeful rather than padded.

Then do the most old-fashioned check of all: read sample answers. Ask a narrow question, a chapter-level question, a cross-chapter synthesis question, a follow-up question, and a concept-link question. Look at the chunks, the retrieval inspection, the reward breakdown, and the final answer. If those examples make sense to a technically literate reader, the system is ready to teach.

31 Appendix F. Extended Foundations And Architecture Notes

These extended reference notes deepen the same Polymind V2 system from a longer-form teaching angle. They are kept in appendix form so the main 23-section spine stays clean while readers who want more architecture, retrieval, RL, evaluation, and workflow detail can keep reading without leaving the handbook.

31.1 F.1 Reading-system foundations

Introduction Section snapshot: This opening section defines what Polymind is, why a deep-reading AI system matters, and what kinds of real-world reading tasks the final build will support. What Is Polymind? Polymind is a reading system, not just a chatbot. Many AI demos can answer questions, but they do not truly help a student work through a long book in a structured way. A serious reading assistant should be able to do more than output a quick paragraph.

It should:

- remember what book it is working with,
- know how the book is organized,
- retrieve the most relevant passages,
- answer with evidence,
- offer summaries at different levels of depth,
- connect ideas across chapters,
- explain concepts in beginner-friendly language,
- improve over time when it sees which answers are more useful.

That is the goal of Polymind. In this guide, Polymind is built as a layered system. Each layer solves a different problem:

Layer	What it does
Why it matters	Book ingestion
	Extracts text from a PDF or .txt file
	The system needs clean raw material
Structure extraction	Detects chapters and sections
	The book is more meaningful when its structure is preserved
Semantic chunking	Breaks the book into useful pieces
Retrieval	works better on focused chunks than on full chapters
Embeddings + FAISS	Turns chunks into searchable vectors
Fast similarity search	is the heart of retrieval
RAG	Combines retrieval with generation
Answers	can stay grounded in the book
RL bandit layer	Scores multiple candidate answers
	The system can choose better answers instead of trusting a single draft
Memory	Keeps track of recent interaction context
Follow-up questions	become more natural

Polymind is especially useful for deep reading tasks such as:

- asking chapter-specific questions,
- comparing two ideas that appear far apart in the book,
- generating study notes,
- producing layered summaries,
- building concept maps,
- exploring themes like apprenticeship, practice, creativity, and mastery.

Why This Project Matters A project like Polymind matters because it teaches both AI engineering and AI thinking.

If you only use an API and send a prompt, you may get a nice answer, but you do not learn how the answer was produced, what evidence supported it, why it might hallucinate, or how to improve it. In

contrast, this project exposes the full chain of decisions: • How do we get text from a book? 2 How do we preserve structure? 3 How do we divide the book into meaningful units? 4 How do we turn text into vectors?

5 How do we find the best evidence for a question? 6 How do we build a prompt that keeps answers grounded? 7 How do we score the quality of several possible answers? 8 How do we remember context across turns? That sequence is far closer to real AI system design than a single prompt call.

That is a feature, not a flaw. Beginners often struggle with AI tutorials because tutorials skip the hidden middle. They show installation, then jump to a working demo, but they do not explain the engineering path in between. In this guide, we will do the opposite. We will explain the intuition first, then write the code, then inspect the outputs, then discuss common mistakes.

Whenever possible, each implementation section follows the same pattern: • Intuition. 2 Colab-ready code cell. 3 Expected outcome. 4 Debugging notes. That makes the project easier to learn, easier to fix, and easier to extend.

Key takeaways: Polymind is framed as a layered reading system, not a one-shot chatbot. The project matters because it teaches retrieval, generation, memory, and RL-based selection as one coherent design. Prerequisites Section snapshot: This section builds the mental model needed for the rest of the guide: Python basics, deep learning intuition, LLMs, RAG, embeddings, and a lightweight view of reinforcement learning. What You Need Before Starting You do not need to be an expert to complete this project, but a few basics help a lot. You should be comfortable with: • running cells in Google Colab, • reading simple Python code, • understanding what a function is, • working with strings, lists, and dictionaries, • copying files into Google Drive.

If some of those feel shaky, that is okay. We will still explain the important parts. A Tiny Python Refresher Python is popular in AI because it is readable and has a rich ecosystem. In this project, you will mostly work with: • variables, • functions, • classes, • lists, • dictionaries, • loops, • dataclasses. Here is a tiny refresher: What Is Deep Learning?

Deep learning is a branch of machine learning built on neural networks. A neural network is a system of layers that transforms input numbers into output numbers. During training, the network adjusts its parameters so the outputs become more useful. In language tasks, the input might be token IDs or embeddings, and the output might be a probability distribution over the next token. Here is the beginner-friendly way to think about it: • ordinary programming tells the computer exactly what rules to use, • deep learning lets the computer learn useful patterns from examples.

For language modeling, those patterns include: • grammar, • phrasing, • common topic structure, • style, • word relationships, • long-range dependencies. What Is an LLM? An LLM, or large language model, is a neural network trained on large amounts of text so it can predict, continue, transform, summarize, or answer based on language. Even though the name sounds mysterious, the core idea is simple: given previous text, predict what text should come next. That training objective is surprisingly powerful.

It allows models to learn: • sentence structure, • question answering patterns, • summarization behavior, • style transfer, • instruction following, • reasoning-like behavior. In this repository, the MiniLLM is a small decoder-only transformer. It is perfect for understanding core mechanisms such as embeddings, attention, normalization, and autoregressive generation. In Polymind, we do not rely on that mini model alone for book QA because a small model trained on a generic corpus is unlikely to answer complex questions about a full nonfiction book well. Instead, we combine: • the mini LLM for education and architectural intuition, • a sentence embedding model for retrieval, • a small instruction model for grounded generation.

What Is RAG? RAG stands for Retrieval Augmented Generation. The idea is straightforward: • A user asks a question. 2 The system retrieves relevant pieces of text from a knowledge source. 3 The generator uses those pieces as context.

31.2 F.2 Architecture and Colab framing

System Architecture Section snapshot: This section maps the full Polymind pipeline from book ingestion to RL-selected answers so the later code always has a clear architectural anchor. The Big Picture Polymind works as a pipeline. The system begins with a book file and ends with a grounded answer. The whole system can be viewed in two linked stages. Why a Layered System Is Better Than a Single Model It is tempting to ask: why not just use one strong language model and skip all this?

There are three reasons. First, books are structured. The system should respect chapter and section boundaries. A single raw prompt does not automatically preserve that structure. Second, groundedness matters.

If the model answers without retrieval, it may sound confident while drifting away from the actual book. Third, answer quality varies. Even grounded prompts can produce different answer styles. A reward layer helps us select the most useful one. The result is not just “more AI.” It is better engineering.

This architecture works in Colab because: • there is only one book, • embeddings are small enough to compute locally, • FAISS is lightweight and fast, • Flan-T5 Base is manageable, • the RL layer is only scoring and bookkeeping, • everything can be saved into Drive as JSON, NumPy arrays, and small model artifacts. Key takeaways: The architecture works because each layer has a narrow job: structure the book, retrieve evidence, generate candidate answers, score them, and preserve useful context in memory. Setting up Google Colab Section snapshot: This section prepares a clean Colab workspace with the required libraries, Drive storage, GPU checks, and reusable configuration objects for the rest of the build. Why Colab Is a Good Fit Google Colab is helpful because it gives you a notebook environment, optional GPU access, and easy integration with Google Drive. That means a student can: • install libraries in a notebook cell, • upload a book file, • store artifacts in Drive, • run the entire pipeline without configuring a local machine.

Install the Libraries Start a fresh Colab notebook and run: Colab Cell !pip install -q sentence-transformers transformers accelerate faiss-cpu pymupdf pandas matplotlib Why these libraries? Library Purpose sentence-transformers semantic embeddings transformers generation model accelerate smoother model loading faiss-cpu vector search pymupdf PDF text extraction pandas evaluation tables matplotlib basic plots Expected outcome: Mount Google Drive

31.3 F.3 Model and transformer background

It is designed to be: • educational, • compact, • trainable on small resources, • easy to inspect. It is not designed to be a strong instruction-following reading assistant for a specific long nonfiction book. That is why Polymind uses: • MiniLLM for understanding the architecture of language models, • all-MiniLM-L6-v2 for retrieval embeddings, • flan-t5-base for practical grounded answer generation. A Tiny Transformer Intuition If you are new to LLMs, the most useful mental model is this: • embeddings turn symbols into vectors, • attention decides what other positions matter, • feed-forward layers transform information inside each position, • repeated blocks gradually build richer representations, • the output head turns representations into token probabilities. If you understand those five points, you already understand the skeleton of a language model.

Mini LLM Configuration in the Repository The current repository uses a small configuration similar to: Parameter Value d_model 256 n_layers 4 n_heads 8 n_kv_heads 2 ff_n_hidden_dim 680 context_length 256 This is enough to demonstrate the machinery, but much smaller than the models we normally use for strong instruction-following behavior. Where MiniLLM Fits into Polymind MiniLLM influences Polymind in three ways: • It teaches the student how language generation works. 2 It motivates why retrieval is needed when context is limited. 3 It makes the “mini LLM” phrase in the project title concrete and honest. The final Polymind system is therefore not anti-LLM.

It is a system that uses LLM ideas responsibly. A Lightweight Concept Mapping Table MiniLLM concept Polymind equivalent Token sequence Book chunk or prompt text Transformer context window Retrieved context budget Next-token distribution Generated answer tokens Sampling settings Answer arm temperature choices Model limitations Reason to use retrieval and RL selection A Useful Takeaway When students first learn LLMs, they often think bigger models solve everything. A better takeaway is: strong AI systems come from good component design, not just bigger parameter counts. That principle is exactly what Polymind demonstrates. Key takeaways: The MiniLLM is the conceptual anchor for transformer learning, not the production QA engine.

Polymind uses that foundation to motivate a hybrid retrieval-based reading system. Book Processing Pipeline Section snapshot: This section turns a raw book file into structured, chapter-aware, semantically meaningful chunks that the rest of the system can search and cite reliably. Why Book Processing Matters Garbage in, garbage out. If the book text is noisy, chapter boundaries are wrong,

or chunks are too large and messy, retrieval quality will suffer. That means even a strong generator will receive weak context.

So we start by treating the book as data that needs careful preparation. Goals of the Processing Pipeline The processing pipeline should do four things well: • Extract readable text from the uploaded file. 2 Remove obvious formatting noise. 3 Recover meaningful structure such as chapters and sections. 4 Produce chapter-aware semantic chunks with metadata.

Extract the Raw Text The primary path uses PDF extraction. The fallback path uses plain text. Clean the Text Book PDFs often contain: • repeated headers, • repeated footers, • page numbers, • too many line breaks, • broken spacing. We will apply conservative cleaning. The goal is to improve readability without accidentally deleting meaning.

Inspect the First Few Thousand Characters Before building automation, inspect the data manually. Detect Chapters Books vary in formatting, so there is no universal chapter detector. For Mastery, a regex-based approach plus a manual correction step is a good balance between automation and reliability. Manual Chapter Correction Cell Because real books are messy, we include a manual override cell. That keeps the project practical.

32 Appendix G. Extended Retrieval, Generation, And RL Notes

These extended reference notes deepen the same Polymind V2 system from a longer-form teaching angle. They are kept in appendix form so the main 23-section spine stays clean while readers who want more architecture, retrieval, RL, evaluation, and workflow detail can keep reading without leaving the handbook.

32.1 G.1 Book processing and chunking details

Clean the Text Book PDFs often contain: • repeated headers, • repeated footers, • page numbers, • too many line breaks, • broken spacing. We will apply conservative cleaning. The goal is to improve readability without accidentally deleting meaning. Inspect the First Few Thousand Characters Before building automation, inspect the data manually. Detect Chapters Books vary in formatting, so there is no universal chapter detector.

For Mastery, a regex-based approach plus a manual correction step is a good balance between automation and reliability. Manual Chapter Correction Cell Because real books are messy, we include a manual override cell. That keeps the project practical. Build Structured Book Sections Now we turn the full text into structured chapter sections. Sentence Splitting We need sentence boundaries because semantic chunking works better when chunks break near natural language boundaries instead of arbitrary character positions.

Semantic Chunking Strategy We want chunks that are: • large enough to preserve context, • small enough to retrieve precisely, • aligned with sentence boundaries, • aware of chapter structure. For this guide, we use: • target chunk size: 350 to 500 words, • overlap: about 20%, • never mix text from different chapters in one chunk. Here is the chunking pipeline: Create Semantic Chunks Save the Chunk Data Saving the chunk data prevents us from repeating preprocessing every session. Inspect Chunk Quality Always inspect a few chunks manually. Why Metadata Is So Important Metadata might feel boring, but it is one of the highest-value parts of the system.

Without metadata, a chunk is just a blob of text. With metadata, a chunk becomes a traceable knowledge unit: • where it came from, • what chapter it belongs to, • what section it belongs to, • how to cite it, • how to group it in summaries, • how to analyze retrieval behavior. This is why each ChunkRecord includes: Field Purpose chapter_id numeric ordering chapter_title human-readable context section_title local structure chunk_id unique citation id text semantic content start_char / end_char source tracing word_count chunk analysis and debugging Common Book Processing

Mistakes Beginners often make the same chunking mistakes: • chunks are too large, so retrieval becomes vague, • chunks are too small, so context is lost, • chunk overlap is zero, so idea transitions break, • chunk overlap is huge, so retrieval becomes repetitive, • chapter boundaries are ignored, so citations become confusing. When in doubt, prefer slightly smaller, cleaner, chapter-aware chunks with moderate overlap. Key takeaways: Better book processing usually leads to better retrieval.

Clean text, honest chapter detection, moderate overlap, and rich metadata create the strongest foundation for grounded answers. Embeddings + Vector Database Section snapshot: This section converts each chunk into a normalized semantic vector and stores those vectors in FAISS so book passages can be searched by meaning instead of keyword matching alone. What Are Embeddings, Really? An embedding is a numeric representation of meaning. When you send a sentence into an embedding model, the model returns a vector, which is simply an ordered list of numbers.

On their own, those numbers do not look intuitive. But in aggregate, they allow the system to place meaningfully similar texts near each other in vector space. That is why embeddings are one of the foundations of modern retrieval. If two texts talk about similar ideas, their vectors often point in similar directions. We can then use cosine similarity or inner product to measure closeness.

Why Use all-MiniLM-L6-v2 ? This guide uses sentence-transformers/all-MiniLM-L6-v2 because it is: • small enough for Colab, • widely used for semantic search, • fast, • easy to load, • good enough for one-book retrieval. This model is not a generator. It does not answer questions directly. Its job is to turn text into semantic vectors.

Load the Embedding Model Turn Chunks into Vectors Save the Embeddings Build the FAISS Index FAISS is a library for fast vector search. In this guide, we use IndexFlatIP, which performs exact inner-product search. That is a good fit because: • our dataset is only one book, • exact search is simple and accurate, • we do not need approximate indexing for this scale. Vector Search Pipeline Why FAISS Instead of a Larger Database? For one book, FAISS is ideal because it is: • simple, • fast, • local, • lightweight.

32.2 G.2 Embeddings, FAISS, and retrieval details

• a hosted vector database, • a cloud search engine, • a large distributed system. That simplicity is good for learning and debugging. A Quick Sanity Check We can test the vector space with a sample query. Key takeaways: Embeddings turn book chunks into searchable semantics, and FAISS turns those vectors into fast retrieval. Together they become the book's searchable memory layer.

Retrieval System Section snapshot: This section explains how Polymind finds the most relevant chunks for a question, then improves those raw results with simple filtering and chapter-aware diversification. Retrieval Is the Core of Grounded QA When users ask questions, Polymind should not guess from memory. It should search the book. The retrieval system answers one crucial question:

which chunks are most useful for answering this query? That is what makes the whole system grounded.

So we need a clean formatter. Retrieval Design Rules These rules are worth remembering: Rule Why it matters Keep chunks semantically meaningful Retrieval is only as good as the chunking Store rich metadata Citations and analysis depend on it Use moderate top_k Too few loses evidence, too many dilutes focus Filter weak results Low-similarity chunks can confuse the generator Inspect retrieval manually Similarity search is not self-explaining A Retrieval Debugging Routine If answers are weak, do this first: • inspect the retrieved chunks, 2 check whether chunk boundaries are sensible, 3 verify that the question is semantically clear, 4 try top_k=8 then diversify back down to 5, 5 compare two similar queries. Many “model problems” are actually retrieval problems. Key takeaways: Retrieval quality drives answer quality. If the system feels vague or hallucinatory, inspecting chunk selection is usually more valuable than blaming the generator first.

RAG Pipeline Section snapshot: This section combines retrieval with generation, showing how prompts turn cited context into readable answers while still keeping the output grounded in the book. From Retrieval to Grounded Generation Retrieval alone gives us passages. Generation turns passages into a readable answer. That combination is RAG: • retrieval provides evidence, • generation provides natural language output. The key challenge is prompt design.

We want the model to answer clearly, but also stay faithful to the retrieved text. Load the Generation Model For this guide we use google/flan-t5-base, a practical text-to-text model that is approachable in Colab. RAG Pipeline Diagram Prompt Design Principles Good RAG prompts usually do three things: • tell the model to use only the provided evidence, 2 tell the model how to structure the answer, 3 tell the model how to cite evidence. We also want the prompt to stay readable so the student can understand it. Build a Grounded Prompt

32.3 G.3 Generation, reward, and policy details

These arms differ by prompt framing and temperature. That gives us meaningful variation without making the system too complex. RL Reward Function Design The reward should capture four things: • relevance to retrieved chunks, 2 semantic similarity, 3 coverage of key ideas, 4 penalty for hallucination. We will use the fixed formula: $\ast \text{semantic_alignment} + \ast \text{keyword_overlap} + \ast \text{key_idea_coverage} - \ast \text{hallucination_penalty}$ Then we clamp the result to the range $[0, 1]$. Let us unpack each term.

Semantic Alignment This measures whether the answer embedding is close to the overall meaning of the retrieved context. If the answer talks about the same central idea as the retrieved chunks, this score should be high. Keyword Overlap This measures whether important content words from the retrieved context also appear in the answer. It is not a perfect measure, but it is helpful because

grounded answers often reuse the book's key language. **Key Idea Coverage** This measures whether the answer touches the top concepts present in the retrieved chunks.

We approximate "top concepts" with the top 10 content keywords from the context. **Hallucination Penalty** This penalizes answers that introduce many unsupported content words not seen in the retrieved context. The goal is not to punish any paraphrasing. The goal is to discourage unsupported drift. **Define RL Data Containers** `average_reward: float = Lightweight Keyword Utilities` To keep the guide beginner-friendly, we use regex tokenization and a small built-in stopword list instead of heavier NLP packages.

Multi-Level Understanding Section snapshot: This section shows how one retrieval stack can support different reading depths, from quick TL;DR outputs to chapter summaries and slower conceptual explanations. **Why One Summary Is Never Enough** Good readers move between levels of abstraction: • quick overview, • chapter summary, • detailed explanation. Polymind should do the same. **Summary Levels We Will Support** Level Output style tldr 3-5 lines chapter 1-2 paragraphs focused on one chapter deep fuller conceptual explanation **Prompt Builder for Summary Levels** `) summary = generate_level_summary( "Summarize Greene's view of practice and persistence.", final_results, text2text, level="deep", ) print(summary)` **Why Multi-Level Outputs Matter** The same reader may want: • a five-line revision summary before class, • a chapter recap while reading, • a detailed conceptual explanation while studying. A strong reading system should adapt to those different needs without rebuilding the whole pipeline.

Key takeaways: Multi-level outputs make Polymind more useful for real study workflows. The same evidence base can support fast revision, chapter review, and deeper explanation. **Memory System Section snapshot:** This section adds conversational continuity by storing recent turns, summarizing older context, and feeding that context back into retrieval and RL-based answer selection. **Why Memory Is Needed** Without memory, the system treats every question as isolated. That feels unnatural when the user asks: • "What does Greene mean by apprenticeship?" • "Now compare that to social intelligence." • "Which chapter emphasizes that more?" The second and third questions depend on earlier context.

Keep the Memory Simple We do not need a complicated long-term memory architecture. For this project, memory can be: • a short summary of older turns, • the last three question-answer turns, • the chunk IDs used in those turns, • the reward summary of the final RL-selected answer. **Memory Data Structure** **A Query Rewriting Helper** Sometimes a follow-up question is vague on its own. A small rewriting step helps. **Why Memory Improves RL** Too Memory does not only help retrieval.

It also helps the RL layer because candidate answers can be scored in a context that better reflects what the user is asking now. For example: • the question may be short, • memory restores missing context, • retrieval becomes more targeted, • candidate answers become more relevant, • reward scoring becomes more meaningful. **Key takeaways:** Memory does not need to be huge to be useful. A short summary plus a few recent turns is enough to make follow-up questions feel grounded and connected. **Question Answering System Section snapshot:** This section combines retrieval, RAG, RL scoring, and memory

into one runnable QA flow that can answer book questions with citations and track which answer strategy won.

A simple strategy is: • retrieve more chunks than usual, 2 diversify by chapter, 3 ask the generator to compare or connect themes, 4 use RL scoring to pick the clearest grounded synthesis. Explain Like I Am 10 This is a powerful teaching mode because it forces simplification without losing the core idea.

33 Appendix H. Extended Evaluation, Optimization, And Assembly Notes

These extended reference notes deepen the same Polymind V2 system from a longer-form teaching angle. They are kept in appendix form so the main 23-section spine stays clean while readers who want more architecture, retrieval, RL, evaluation, and workflow detail can keep reading without leaving the handbook.

33.1 H.1 Evaluation and acceptance guidance

Evaluation Section snapshot: This section measures whether retrieval is relevant, whether answers stay grounded, and whether the RL layer actually improves answer quality over a baseline RAG path. Why Evaluation Matters An AI system that “seems good” is not the same as a system we understand. Evaluation helps us answer: • is retrieval bringing the right evidence? • is the generator staying grounded? • is RL actually improving answer selection?

• is memory helping or hurting? Without evaluation, improvement becomes guesswork. Three Evaluation Layers Layer Question Retrieval Did we fetch useful chunks? Generation Is the answer clear and grounded? RL policy Is answer selection improving over time?

A Small Evaluation Set Create at least 15 questions that span the main behaviors of the system: Retrieval Quality Check For each question, inspect whether the top retrieved chunks are actually relevant. A simple manual rubric: Score Meaning 2 highly relevant • partially relevant 0 weak or irrelevant If average retrieval quality is low, fix chunking or retrieval before blaming generation. Groundedness and Completeness Rubric Use this simple answer rubric: Criterion What to ask Groundedness Does the answer stay close to retrieved evidence? Completeness Does it answer the full question? Clarity Is it easy to understand?

The reward function is a useful heuristic, not ground truth. Acceptance Checklist Your system is in good shape if: • retrieval returns meaningful passages, • answers include citations, • use_rl=True often scores better than baseline, • memory helps follow-up questions, • outputs remain understandable for a beginner. Key takeaways: Evaluation turns intuition into evidence. Retrieval rubrics, groundedness checks, and baseline-vs-RL comparisons make the system easier to trust and improve. Optimization Section snapshot: This section focuses on practical improvements that make Polymind faster, cleaner, and easier to operate without changing the underlying architecture.

Where the Time Goes Most latency in Polymind comes from: • embedding the book the first time, • generating answers, • evaluating multiple candidate answers. That is why optimization should focus on: • caching, • chunk quality, • prompt efficiency, • RL efficiency. Practical Speed Improvements Optimization Effect save chunk embeddings avoids repeated encoding save FAISS index avoids repeated indexing reduce top_k slightly shorter prompts keep chunk size moderate cleaner retrieval use GPU if available faster generation and embedding evaluate fewer arms later lower RL compute cost

33.2 H.2 Optimization and future improvements

Better Chunking Often Beats Bigger Models This is one of the most important lessons in RAG: better retrieval data often improves answers more than a slightly bigger generator. If the system struggles, revisit: • chapter detection, • overlap size, • section boundaries, • repeated formatting noise, • weak metadata. **Prompt Optimization Ideas** You can often improve output by tightening the prompt: • explicitly ask for citations, • explicitly say “use only the retrieved context,” • keep answer structure simple, • use distinct prompt styles for each arm. **RL Optimization Ideas** Later, if you want faster inference, you can: • let the bandit choose only • or 2 arms most of the time, • generate the preferred arm first, • skip low-value arms after enough history, • store per-question-type statistics. For this guide, full three-arm evaluation is kept because it is easier to inspect and understand.

Stronger Reinforcement Learning This guide uses a bandit because it is the right educational balance. Future work could explore: • human feedback collection, • pairwise preference learning, • reward-model training, • policy optimization beyond bandits. **Tool Usage and Agents** Polymind could also grow into a tool-using system: • timeline extraction, • note export, • flashcard generation, • chapter quiz generation, • concept graph drawing. **Fine-Tuning and Domain Adaptation** Another future path is domain adaptation: • fine-tune a small summarizer, • train a better reranker, • build question-type classifiers, • tune reward weights using human judgments. The important thing is to see future work as layered improvement, not total redesign.

Key takeaways: The future roadmap is additive. Multi-book search, stronger reranking, richer feedback loops, and new tools can all grow from the same core system design. **Final Project Assembly** Section snapshot: This closing section assembles the entire system behind one clean PolymindSystem interface so the project feels like a polished application rather than a loose set of notebook fragments. **Put Everything Behind One Interface** A project feels complete when the parts come together behind a clean API. That is why we wrap the system into one PolymindSystem class.

33.3 H.3 Final assembly and workflow recap

It shows that modern AI is not only about model size. It is about system design, evidence grounding, and careful iteration. Next Steps for the Student If you want to keep going after this guide, try one improvement at a time: • tune chunk size and overlap, 2 test different retrieval settings, 3 improve prompt templates, 4 analyze which RL arm performs best, 5 add a second book, 6 export notes or flashcards, 7 try a stronger generator. That step-by-step mindset will help you extend Polymind without losing clarity. Closing Thought Polymind began as a reading system for one book.

But the ideas you learned here scale far beyond that one example. The same architecture pattern can help you build research assistants, study tools, knowledge systems, and grounded AI products that are both more useful and more trustworthy. That is an excellent place to continue learning. Key takeaways: The final system now combines RAG, memory, and RL-based answer optimization in one coherent workflow. More importantly, it shows how premium AI systems emerge from careful structure, evidence grounding, and iterative design.